

Chapter 4: The Processor

Pipelining, Hazards, and Control

Combined Notes

Contents

1 Introduction to Pipelining

Pipelining is an implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.

- **Core Concept:** Just as a car assembly line works on multiple cars simultaneously (one getting an engine, another getting wheels), a pipelined processor works on different stages of multiple instructions at the same time.
- **Goal:** To increase instruction **throughput**.
- **Throughput:** The number of instructions completed per unit of time.
- **Latency:** The time required to complete a single instruction.

The Pipelining Paradox

Pipelining does **NOT** decrease the time it takes to execute a single instruction (latency). In fact, it often slightly increases it due to the overhead of pipeline control and registers. However, because we are finishing more instructions per second, the **total execution time for a program decreases**.

1.1 The Laundry Analogy

Lecture 4.5 uses a laundry analogy to explain pipelining. Imagine you have four loads of laundry (A, B, C, D) and four stages: **Wash, Dry, Fold, Store**. Each stage takes 30 minutes.

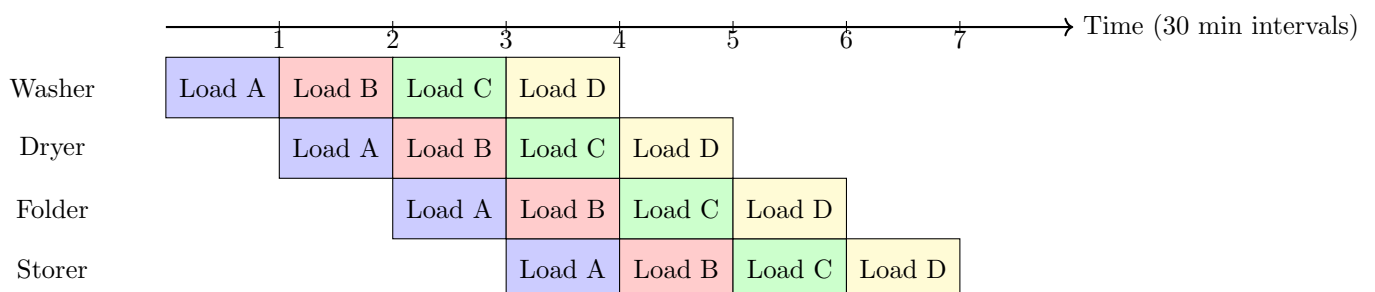
1.1.1 Sequential vs. Pipelined Execution

1. Sequential (Non-Pipelined): You wait for Load A to finish all 4 stages (2 hours) before starting Load B.

- Load A: 6:00 - 8:00
- Load B: 8:00 - 10:00 ...
- **Total Time for 4 loads:** 4×2 hours = 8 hours.

2. Pipelined:

- **6:00:** Load A into Washer.
- **6:30:** Move Load A to Dryer. Put Load B into Washer.
- **7:00:** Move A to Fold, B to Dryer. Put C into Washer.
- **Total Time for 4 loads:** 3.5 hours.



Note: At Time=3.5, all stages are busy!

Figure 1: Visualizing Pipelined Laundry (Ideal Case)

2 The classic Five-Stage RISC-V Pipeline

Most RISC-V implementations use a 5-stage pipeline.

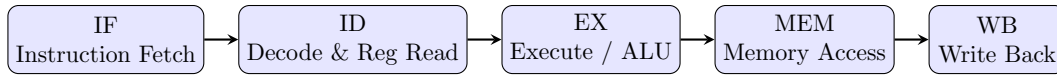


Figure 2: The 5 Stages of RISC-V Pipeline

1. **IF (Instruction Fetch):** Fetch the instruction from memory using the Program Counter (PC).
2. **ID (Instruction Decode):** Decode the instruction and read source registers from the Register File.
3. **EX (Execute):** Perform the operation (ALU) or calculate a memory address.
4. **MEM (Memory Access):** Read/Write data from/to Data Memory (only for Loads/Stores).
5. **WB (Write Back):** Write the result back into the Register File.

Performance Math: Speedup Formula

If all stages are perfectly balanced, the potential speedup is equal to the **number of pipeline stages**.

$$\text{Speedup} = \frac{\text{Pipeline Depth}}{1 + \text{Pipeline Stalls}} \quad (1)$$

In our ideal laundry example with 4 stages, the speedup approaches 4x as the number of loads increases.

- **Single-Cycle Clock:** Must be long enough for the *slowest instruction* (e.g., 800ps).
- **Pipelined Clock:** Must be long enough for the *slowest stage* (e.g., 200ps).

3 Design for Pipelining

Why is RISC-V so good for pipelining compared to x86?

- **Fixed Length (32-bit):** IF and ID stages are simple and predictable. We know where the next instruction is immediately ($PC + 4$).
- **Regular Formats:** Source registers (**rs1**, **rs2**) are always in the same bit positions. We can read registers *while* decoding.
- **Load-Store Architecture:** Calculation and Memory Access are separated. We don't have complex instructions like `ADD [Memory], Register` that would require a complex loop in the pipeline.

4 Pipeline Hazards

A **Hazard** is why we can't always just pipeline everything perfectly. It prevents the next instruction from executing in the next cycle.

4.1 1. Structural Hazards

Problem: Hardware resource conflict.

Analogy: You only have one robotic arm that does both putting clothes in the washer AND folding. It can't do both at once.

Computer Example: If instruction memory and data memory were the same single block, we couldn't Fetch (IF) an instruction while another instruction Accesses Memory (MEM).

Solution: Separate Instruction Memory (IMem) and Data Memory (DMem), or separate L1 caches.

4.2 2. Data Hazards

Problem: An instruction needs data that hasn't been written back yet.

Example Code:

```
add x19, x0, x1    // Produces a value for x19
sub x2, x19, x3    // Needs that value of x19 IMMEDIATELY
```

The `sub` instruction is in the ID stage while `add` is only in EX. The value won't be written to the register file until `add` reaches WB.

4.2.1 Solution A: Forwarding (Bypassing)

Don't wait for the data to be written to the register! Grab it directly from the ALU output buffer.

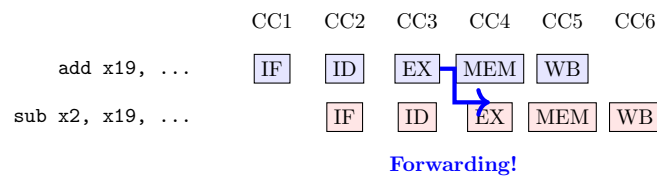


Figure 3: Visualizing Forwarding: Data goes from ADD's EX directly to SUB's EX.

4.2.2 Solution B: Stalling (Load-Use Hazard)

If the producing instruction is a **Load**, the data isn't ready until after MEM. Forwarding from EX isn't possible because the data isn't there yet! We **must** wait one cycle. This is a **Bubble**.

4.3 3. Control Hazards

Problem: Deciding which instruction to fetch next.

Context: Conditional branches (`beq`, `bne`). The processor fetches the next instruction *before* it knows if the branch is taken or not.

Solution:

- **Stall:** Wait until outcome is known (insert bubbles). Simple but slow.
- **Predict:** Guess the outcome (e.g., "Always Not Taken").
 - If right: No penalty.
 - If wrong: Flush pipeline (cancel instructions), restart.
- **Dynamic Prediction:** Hardware remembers recent behavior (Taken/Not Taken) to guess future behavior. Accuracy > 90%.
- **Delayed Branch (MIPS):** The architecture automatically executes the instruction *immediately following* the branch (in the "delay slot") regardless of the branch outcome. The compiler fills this slot with a safe instruction.

5 Flashcards

1. **Q: What is pipelining in the context of computer architecture?**
A: It is an implementation technique in which multiple instructions are overlapped in execution, much like an assembly line.
2. **Q: What is the primary performance goal of pipelining?**
A: Pipelining aims to improve instruction throughput, which is the number of instructions completed per unit of time.
3. **Q: What is the 'pipelining paradox' regarding instruction execution time?**
A: The time to complete a single instruction (latency) is not shorter, but the total time for many instructions is shorter due to increased throughput.
4. **Q: Under ideal conditions with a large number of instructions, the speed-up from pipelining is approximately equal to what?**
A: The number of pipe stages.
5. **Q: What are the five classic stages of a RISC-V instruction pipeline?**
A: 1. Fetch instruction, 2. Decode instruction/Read registers, 3. Execute, 4. Access memory, 5. Write back result.
6. **Q: In a pipelined processor, what determines the duration of the clock cycle?**
A: The duration of the slowest pipeline stage.
7. **Q: In a non-pipelined, single-cycle processor, what determines the duration of the clock cycle?**
A: The time required to execute the slowest possible instruction.
8. **Q: What is a pipeline hazard?**
A: A situation in pipelining when the next instruction cannot execute in the following clock cycle for some reason.
9. **Q: Events in a pipeline such as start-up and ----- affect performance when the number of tasks is not large compared to the number of stages.**
A: wind-down
10. **Q: What is a structural hazard in a pipeline?**
A: A situation where a planned instruction cannot execute because the hardware cannot support the combination of instructions set to execute in the same clock cycle.
11. **Q: What is a data hazard in a pipeline?**
A: A situation where an instruction cannot execute in the proper clock cycle because data it needs is not yet available from a previous instruction.
12. **Q: What is a control hazard in a pipeline?**
A: A situation where the proper instruction cannot execute because the instruction that was fetched is not the one that is needed, typically due to a branch.
13. **Q: What is another common name for a control hazard?**
A: A branch hazard.
14. **Q: How does having a fixed length for all RISC-V instructions simplify pipelining?**
A: It makes it much easier to fetch instructions in the first pipeline stage and decode them in the second stage.
15. **Q: How does RISC-V being a load-store architecture simplify pipelining?**
A: It allows the memory address calculation and memory access to occur in predictable, separate stages (EX and MEM).
16. **Q: A specific type of data hazard where data from a load instruction is needed by a subsequent instruction before it's available is called a -----.**
A: load-use data hazard
17. **Q: What is forwarding, also known as bypassing, used for?**
A: It is a method of resolving a data hazard by retrieving the missing data from internal buffers rather than waiting for it to be written to the register file.

18. **Q: Why is forwarding sometimes insufficient, such as in a load-use data hazard?**
A: The desired data is not available until after the memory access stage, which is too late for the execution stage of the immediately following instruction.
19. **Q: A stall initiated in order to resolve a hazard is often called a ____.**
A: bubble
20. **Q: Besides hardware stalls, what is a software technique a compiler can use to mitigate data hazards?**
A: Reordering code to separate a dependent instruction from the one that produces its data.
21. **Q: What is the simplest, albeit slow, hardware solution to a control hazard caused by a branch?**
A: To stall the pipeline after fetching a branch until the branch outcome and target address are known.
22. **Q: A method of resolving a branch hazard by assuming an outcome and proceeding rather than waiting is known as ____.**
A: branch prediction
23. **Q: What is a simple static branch prediction strategy mentioned in the text?**
A: To predict that conditional branches will always be untaken.
24. **Q: What is dynamic branch prediction?**
A: A technique where hardware makes predictions based on the past behavior of each individual conditional branch, and may change predictions during program execution.
25. **Q: What must the pipeline control do when a branch prediction is incorrect?**
A: It must ensure the wrongly fetched instructions have no effect and restart the pipeline from the correct branch address.
26. **Q: An architectural solution to control hazards, used by MIPS, that always executes the instruction immediately following a branch is called a ____.**
A: delayed branch
27. **Q: Pipelining improves instruction ____ rather than instruction ____.**
A: throughput, latency
28. **Q: What is pipeline latency?**
A: The number of stages in a pipeline, representing the time it takes for a single instruction to complete its execution.
29. **Q: In the RISC-V single-cycle example, the clock cycle was 800 ps because it had to be stretched to accommodate the slowest instruction, which was ____.**
A: ld (load doubleword)
30. **Q: In the graphical representation of forwarding in Figure 4.27, the path goes from the output of the EX stage of the add instruction to the input of the EX stage of which instruction?**
A: The sub instruction.
31. **Q: In the code reordering example, which instruction was moved up to eliminate the pipeline stalls?**
A: The third ld instruction (ld x4, 16(x31)).
32. **Q: Why is it impossible for a forwarding path to go from the MEM stage of one instruction to the EX stage of the *following* instruction without a stall?**
A: This would mean going backward in time, as the following instruction's EX stage occurs at the same time as the current instruction's MEM stage.
33. **Q: Assuming conditional branches are 17% of instructions, what is the CPI if every branch causes a one-cycle stall and all other instructions have a CPI of 1?**
A: The CPI would be 1.17.
34. **Q: In a pipeline diagram, what does shading on the right half of the register file symbol indicate?**
A: It indicates that the register file is being read in that stage.

35. **Q: In a pipeline diagram, what does shading on the left half of the register file symbol indicate?**
A: It indicates that the register file is being written in that stage.
36. **Q: The text states that modern implementations of the x86 architecture often translate complex x86 instructions into simpler, RISC-V-like operations before pipelining them. Why is this done?**
A: Because x86 instructions vary in length from 1 to 15 bytes, making direct pipelining considerably more challenging.
37. **Q: In what pipeline stage does a RISC-V instruction write its result?**
A: The last stage (Write-back or WB).
38. **Q: In the example from Figure 4.24, what was the single-cycle clock time and what was the pipelined clock time?**
A: The single-cycle time was 800 ps (for the slowest instruction, ld), and the pipelined time was 200 ps (for the slowest stage, memory access or ALU).
39. **Q: What is a classic example of a structural hazard in a processor pipeline?**
A: When one instruction is accessing data memory while another instruction is being fetched from memory, if there is only a single memory unit.
40. **Q: In the sequence add x19, x0, x1 followed by sub x2, x19, x3, how does forwarding resolve the data hazard?**
A: The result from the ALU in the EX stage of the add instruction is forwarded directly to the ALU input for the EX stage of the sub instruction.
41. **Q: A forwarding path is only valid if the destination stage is ----- in time than the source stage.**
A: later
42. **Q: In the example $a = b + e$; $c = b + f$;, how can the code be reordered to avoid pipeline stalls?**
A: By moving the load for f up, between the loads for b and e and the first add instruction, to separate the dependent instructions.
43. **Q: What happens in a 'predict-not-taken' scheme if the branch is actually taken?**
A: The pipeline stalls, and the incorrectly fetched instructions are flushed while the pipeline restarts from the correct branch address.
44. **Q: What is the key difference between static and dynamic branch prediction?**
A: Static prediction is rigid (e.g., always predict taken for backward branches), while dynamic prediction uses the past behavior of each specific branch to make its guess.
45. **Q: What is the typical accuracy of modern dynamic branch predictors?**
A: They can correctly predict conditional branches with more than 90% accuracy.
46. **Q: For the code sequence: ld x10, 0(x10) followed by add x11, x10, x10, can it execute without stalling?**
A: No, this is a load-use hazard that must stall for one cycle, even with forwarding.
47. **Q: For the code sequence: add x11, x10, x10 followed by addi x12, x11, 5, can it execute without stalling?**
A: Yes, this data hazard can be resolved using only forwarding.
48. **Q: For the code sequence: addi x11, x10, 1 followed by addi x12, x10, 2 and addi x13, x10, 3, can it execute without issues?**
A: Yes, it can execute without stalling or forwarding because there are no data dependencies between the instructions.
49. **Q: If one stage in a pipeline is slower than the others, what effect does it have?**
A: The slowest stage determines the clock cycle time for the entire pipeline.
50. **Q: Ideally, what is the speedup from pipelining equal to?**
A: The number of stages in the pipeline.

51. **Q: Why do x86 architectures translate their instructions before pipelining?**
A: Because x86 instructions vary in length from 1 to 15 bytes, which makes standard pipelining very difficult; they are translated into simpler, fixed-length micro-operations.
52. **Q: In the structural hazard example with a single memory, which two pipeline stages collide?**
A: The IF stage (fetching instruction $i + 3$) and the MEM stage (accessing data for instruction i).
53. **Q: In the code reordering example ($a = b + e; c = b + f$), how many cycles are saved by moving the load for 'f' up?**
A: 2 cycles (it eliminates two 1-cycle stalls).
54. **Q: What is the colloquial nickname for a pipeline stall mentioned in the text?**
A: A "bubble".
55. **Q: What is the specific calculated CPI penalty for "Stall on Branch" if 17% of instructions are branches?**
A: 1.17 (Calculated as $1 + 0.17 \times 1$ stall cycle).
56. **Q: Which specific architecture uses the "Delayed Branch" technique where the instruction after a branch is always executed?**
A: The MIPS architecture.
57. **Q: In a Delayed Branch system, where does the compiler place a "safe" instruction?**
A: Immediately after the branch instruction (in the branch delay slot).
58. **Q: What does the text say about the visibility of pipelining to the programmer?**
A: It is fundamentally invisible to the programmer (unlike multiprocessor programming).
59. **Q: Define 'Instruction Throughput'.**
A: The quantitative measure of the number of instructions completed per unit of time by a processor.
60. **Q: Define 'Pipeline Latency'.**
A: The time interval required for a single instruction to traverse all stages of the pipeline and complete execution.
61. **Q: What is a 'Pipeline Stall' (Bubble)?**
A: A temporary suspension of instruction fetching and execution, effectively inserting NOPs (No Operations), to resolve a hazard.
62. **Q: Clearly distinguish between 'Forwarding' and 'Bypassing'.**
A: They are synonymous terms. Both refer to the hardware technique of routing a data result from an internal pipeline buffer (e.g., ALU output) directly to a functional unit input, skipping the Register File write-read cycle.
63. **Q: What technically constitutes a 'Load-Use Data Hazard'?**
A: A specific data dependency where an instruction requires an operand that is being loaded from memory by the immediately preceding instruction. The data is not available in the pipeline until the MEM stage completes.
64. **Q: What is 'Branch Prediction Miss Penalty'?**
A: The number of clock cycles wasted when a branch outcome is mispredicted, requiring the pipeline to be flushed and refilled from the correct target address.
65. **Q: Define 'Static Branch Prediction'.**
A: A prediction strategy that uses fixed heuristics (e.g., "backward branches are usually taken") and does not change based on execution history.
66. **Q: Define 'Dynamic Branch Prediction'.**
A: A prediction strategy that uses hardware structures (like Branch History Tables) to track the past outcomes of branches and predict future behavior based on that history.
67. **Q: technically, why does a 'Control Hazard' occur?**
A: Because the Program Counter (PC) must be updated in the IF stage, but the condition for a branch instruction is often not resolved until the ID or EX stage.

68. Q: What is the 'Branch Delay Slot' in MIPS architecture?

A: The instruction position immediately following a branch instruction. This instruction is executed regardless of the branch outcome to utilize the cycle that would otherwise be a stall.

69. Q: What is the function of the 'IF/ID Register'?

A: A pipeline register that holds the instruction fetched from memory and the PC value, preserving them for the Decode stage while the Fetch stage retrieves the next instruction.

70. Q: What is 'PC-Relative Addressing' in the context of branches?

A: A method where the branch target address is calculated by adding an immediate offset to the current Program Counter (PC) value.

71. Q: In RISC-V, what specific hazard is NOT resolved by forwarding alone?

A: The Load-Use Data Hazard. It requires a one-cycle stall even with forwarding.

6 Introduction: Pipelining the Single-Cycle Datapath

“There is less in this than meets the eye.” – Tallulah Bankhead

The division of an instruction into five stages means a five-stage pipeline. Consequently, up to **five instructions** will be in execution during any single clock cycle.

6.1 The Five Stages

We separate the datapath into five pieces, named after the stage of execution:

1. **IF**: Instruction Fetch
2. **ID**: Instruction Decode and Register File Read
3. **EX**: Execution or Address Calculation
4. **MEM**: Data Memory Access
5. **WB**: Write Back

6.2 Flow of Data

Instructions and data generally move from **left to right** through these five stages. They get “cleaner, drier, and more organized” as they move through the line (Laundry Analogy).

Exceptions to Left-to-Right Flow

There are two critical exceptions where data flows **right-to-left**:

1. **Write-Back Stage (WB)**: Places the result back into the Register File in the middle of the datapath (Stage 5 → Stage 2). This causes *Data Hazards*.
2. **PC Update**: Selecting the next PC value (Branch address from MEM → IF). This causes *Control Hazards*.

7 Pipeline Registers

To share a single datapath among multiple instructions without them interfering, we must place **Pipeline Registers** between each stage. These act like “baskets” in the laundry analogy, holding data for the next stage.

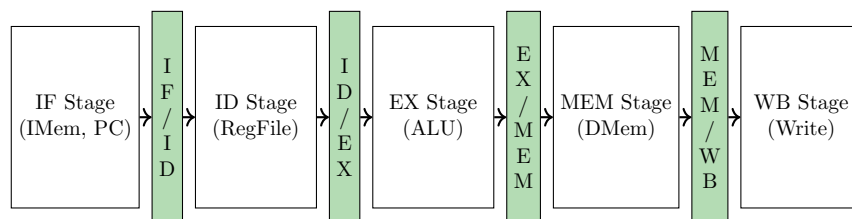


Figure 4: Pipelined Datapath with Pipeline Registers (Highlighted in Green)

7.1 Register Sizing

These registers must be wide enough to store **ALL** data needed for subsequent stages.

- **IF/ID (96 bits)**: Holds 32-bit Instruction + 64-bit PC.
- **ID/EX (256 bits)**: Holds Register Values, Immediate, PC, etc.
- **EX/MEM (193 bits)**: Holds ALU Result, Write Data, Dest Reg, etc.
- **MEM/WB (128 bits)**: Holds Memory Data, ALU Result, Dest Reg.

Note

There is **no pipeline register** at the end of WB because the state (Register File, Memory, PC) is updated directly.

8 Instruction Walkthroughs

8.1 1. Load Instruction (ld)

Active in all 5 stages.

- **IF:** Fetch instruction, increment PC. Save to IF/ID.
- **ID:** Read registers, sign-extend immediate. Save to ID/EX.
- **EX:** ALU adds register + immediate (address calculation). Save to EX/MEM.
- **MEM:** Read data memory using address. Save to MEM/WB.
- **WB:** Write loaded data into Register File.

8.2 2. Store Instruction (sd)

- **IF & ID:** Same as Load.
- **EX:** Calculate address. **Crucially**, we must also pass the value to be stored (read in ID) from ID/EX to EX/MEM.
- **MEM:** Write data to memory.
- **WB: Idle.** Nothing happens.

8.3 3. R-Type Instruction (e.g., add, sub)

Although 4.6.txt primarily uses Load/Store to illustrate the datapath, R-Types are simpler:

- **IF & ID:** Same as Load.
- **EX:** ALU performs operation. Result to EX/MEM.
- **MEM: Idle.** Result passes through.
- **WB:** Result written to Register File.

9 The Design “Bug” & Structural Hazard

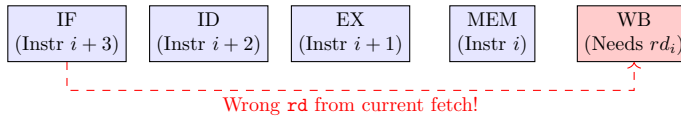
The Bug: Destination Register

In the native breakdown, the instruction is decoded in **ID**, identifying the destination register (**rd**). However, the write occurs in **WB**.

The Problem: By the time the Load instruction reaches WB (5 cycles later), the IF stage is fetching a *different* instruction. WB would use the wrong register bits!

The Fix: We must **preserve the destination register number** by passing it along the pipeline registers.

A. The Bug (Wrong)



B. The Fix (Correct)



Figure 5: Visualizing the Bug (Top) and the Fix (Bottom).

10 Visualizing Pipelines

10.1 1. Multi-Clock-Cycle Diagram (Traditional Style)

Shows execution over time with detailed stage activities (similar to Figure 4.42).

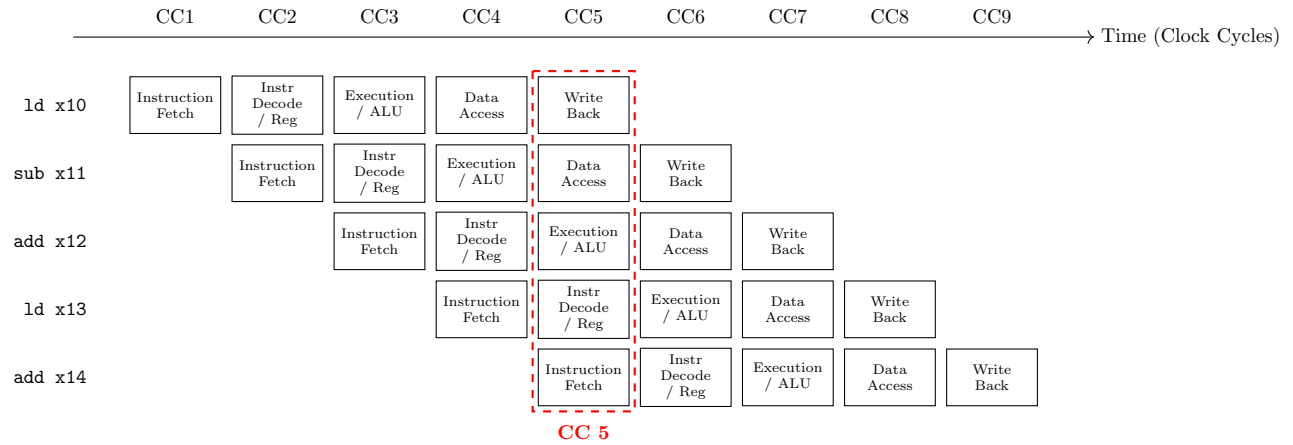
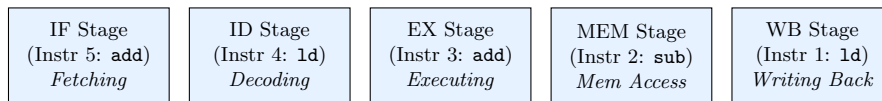


Figure 6: Traditional Multiple-Clock-Cycle Pipeline Diagram. In Cycle 5, each instruction is in a different stage.

10.2 2. Single-Clock-Cycle Diagram (Vertical Slice)

Represents the state of the processor at a specific instant (e.g., Cycle 5).



At Cycle 5, the hardware is fully utilized, handling 5 different instructions simultaneously without conflict (assuming no hazards).

Figure 7: Single-Clock-Cycle Diagram: A snapshot of the processor state at Cycle 5.

11 Pipelined Control

Control lines are just like single-cycle, but applied at different times. We group them by stage and pass them through pipeline registers.

Stage	Control Lines	Function
IF	(None)	standard read/write
ID	(None)	Fixed decoding
EX	ALUOp, ALUSrc	Execution Control
MEM	Branch, MemRead, MemWrite	Memory/Branching
WB	MemtoReg, RegWrite	Register Write

11.1 Control Signal Propagation

Control signals are generated in the **ID stage** but used in later stages (EX, MEM, WB). To ensure they arrive at the correct time, they are passed through the pipeline registers:

1. **ID/EX Register:** Holds all control signals for EX, MEM, and WB.
2. **EX/MEM Register:** Passes MEM and WB signals.
3. **MEM/WB Register:** Passes WB signals.

12 Check Yourself (from text)

1. **True/False:** allowing instructions to take fewer stages always increases performance?
False. Throughput is determined by the clock cycle time (slowest stage), not individual instruction latency.
2. **True/False:** Pipeline stages affect latency, not throughput?
True.
3. **True/False:** You can make branches take fewer cycles?
True, but it complicates control.
4. **True/False:** Making the pipeline longer (more stages) can improve performance?
True, if it allows for a shorter clock cycle (higher frequency).

13 Flashcards

1. **Q: What is the first stage of the five-stage pipeline, and what is its abbreviation?**
A: The first stage is Instruction Fetch, abbreviated as IF.
2. **Q: What is the second stage of the five-stage pipeline, and what is its abbreviation?**
A: The second stage is Instruction Decode and register file read, abbreviated as ID.
3. **Q: What is the third stage of the five-stage pipeline, and what is its abbreviation?**
A: The third stage is Execution or address calculation, abbreviated as EX.
4. **Q: What is the fourth stage of the five-stage pipeline, and what is its abbreviation?**
A: The fourth stage is Data memory access, abbreviated as MEM.
5. **Q: What is the fifth stage of the five-stage pipeline, and what is its abbreviation?**
A: The fifth stage is Write back, abbreviated as WB.
6. **Q: In a five-stage pipeline, up to how many instructions can be in execution during a single clock cycle?**
A: Up to five instructions can be in execution during any single clock cycle.
7. **Q: What is the general direction of flow for instructions and data through the pipeline stages?**
A: Instructions and data generally move from left to right through the five stages.
8. **Q: What is the first exception to the left-to-right flow of data in the pipelined datapath?**
A: The write-back stage, which sends the result back to the register file in the middle of the datapath.
9. **Q: What is the second exception to the left-to-right flow of data in the pipelined datapath?**
A: The selection of the next PC value, which chooses between the incremented PC and a branch address from the MEM stage.
10. **Q: The right-to-left data flow of the write-back stage can lead to what type of hazard?**
A: This flow can lead to data hazards.
11. **Q: The right-to-left data flow for selecting the next PC value can lead to what type of hazard?**
A: This flow can lead to control hazards.
12. **Q: To share a single datapath among multiple instructions in a pipeline, what hardware components must be added between stages?**
A: Pipeline registers must be added to hold data between stages.
13. **Q: What is the naming convention for pipeline registers?**
A: They are named for the two stages they separate (e.g., the register between IF and ID is named IF/ID).
14. **Q: Why is there no pipeline register at the end of the write-back (WB) stage?**
A: It is redundant because all instructions must update some processor state (register file, memory, or PC) which already holds the result.
15. **Q: Which component of the processor's visible architectural state can be thought of as a pipeline register that feeds the IF stage?**
A: The Program Counter (PC) can be thought of as a pipeline register feeding the IF stage.
16. **Q: What two pieces of information are placed into the IF/ID pipeline register during the Instruction Fetch stage?**
A: The fetched instruction and the incremented PC address are placed in the IF/ID register.
17. **Q: During the ID stage of a load instruction, what values are stored in the ID/EX pipeline register?**
A: The two read register values, the sign-extended immediate value, and the PC address are stored in the ID/EX register.
18. **Q: What calculation is performed by the ALU during the EX stage of a load ('ld') instruction?**
A: The ALU adds the contents of a register and the sign-extended immediate to calculate the effective memory address.

19. **Q: During the MEM stage of a load ('ld') instruction, data is read from memory and placed into which pipeline register?**
A: The data read from memory is placed into the MEM/WB pipeline register.
20. **Q: What is the final action performed during the WB stage of a load ('ld') instruction?**
A: Data is read from the MEM/WB pipeline register and written into the register file.
21. **Q: For a store instruction, what is passed from the ID/EX pipeline register to the EX/MEM pipeline register to be used in the MEM stage?**
A: The register data to be stored is passed into the EX/MEM pipeline register.
22. **Q: What is the primary action during the MEM stage of a store ('sd') instruction?**
A: The data from the EX/MEM pipeline register is written into data memory.
23. **Q: What happens during the write-back (WB) stage for a store ('sd') instruction?**
A: Nothing happens in the write-back stage for a store instruction.
24. **Q: To prevent a structural hazard, each logical component of the datapath (e.g., ALU, data memory) can be used within how many pipeline stages?**
A: Each logical component can be used only within a single pipeline stage.
25. **Q: What was the design bug in the initial pipelined datapath concerning the load ('ld') instruction?**
A: The write register number was not passed down the pipeline, so the wrong instruction would supply it during the WB stage.
26. **Q: How was the design bug for the load ('ld') instruction fixed?**
A: The destination register number is passed from the ID stage through the ID/EX, EX/MEM, and MEM/WB pipeline registers for use in the WB stage.
27. **Q: What is the key difference between a multiple-clock-cycle pipeline diagram and a single-clock-cycle pipeline diagram?**
A: A multiple-clock-cycle diagram shows instruction execution over time, while a single-clock-cycle diagram shows the state of the entire datapath at one specific clock cycle.
28. **Q: A single-clock-cycle diagram represents a _____ slice of a multiple-clock-cycle diagram.**
A: vertical
29. **Q: Why does allowing some instructions to take fewer cycles not necessarily improve pipeline throughput?**
A: Throughput is determined by the clock cycle, not the latency (number of stages) of individual instructions.
30. **Q: Instead of making instructions take fewer cycles, what alternative approach could improve pipeline performance?**
A: Making the pipeline longer so instructions take more, but shorter, clock cycles could improve performance.
31. **Q: Control signals for the EX, MEM, and WB stages are created during which pipeline stage?**
A: The control signals are created during the instruction decode (ID) stage.
32. **Q: How are control signals transported to the correct pipeline stage for execution?**
A: They are passed down the pipeline by extending the pipeline registers (ID/EX, EX/MEM, MEM/WB) to include control bits.
33. **Q: Which two control lines are set in the Execution/address calculation (EX) stage?**
A: The ALUOp and ALUSrc control lines are set in the EX stage.
34. **Q: Which three control lines are set in the Memory access (MEM) stage?**
A: The Branch, MemRead, and MemWrite control lines are set in the MEM stage.
35. **Q: Which two control lines are set in the Write-back (WB) stage?**
A: The MemtoReg and RegWrite control lines are set in the WB stage.
36. **Q: The control signals created in the ID stage are first placed into which pipeline register?**
A: They are placed into the ID/EX pipeline register.

37. **Q: Why is an instruction passed through a pipeline stage even if there is nothing for it to do in that stage?**
A: This is done because later instructions are already progressing at the maximum rate, and there is no way to accelerate them.
38. **Q: What information must be preserved and passed down the pipeline for a load instruction to function correctly?**
A: The destination register number must be preserved and passed through the pipeline registers to the WB stage.
39. **Q: In the stylized datapath diagrams, the register file is broken into two logical parts to represent its dual use in which two stages?**
A: It is used for register reads during the ID stage and register writes during the WB stage.
40. **Q: What is the purpose of the ALUSrc control signal in the EX stage?**
A: It selects whether the second ALU input is from a register (Read data 2) or the sign-extended immediate.
41. **Q: What is the purpose of the MemtoReg control signal in the WB stage?**
A: It decides whether the value written to the register file comes from the ALU result or from data memory.
42. **Q: What is the primary activity in the Instruction Fetch (IF) stage of a pipeline?**
A: Reading an instruction from memory using the address in the Program Counter (PC).
43. **Q: What are the two main activities in the Instruction Decode (ID) stage of a pipeline?**
A: Decoding the instruction and reading the contents of the specified registers from the register file.
44. **Q: What is the primary activity in the Execution (EX) stage of a pipeline?**
A: Performing an operation in the ALU, such as an arithmetic calculation or an address calculation.
45. **Q: What is the primary activity in the Data Memory Access (MEM) stage of a pipeline?**
A: Reading from or writing to the data memory.
46. **Q: What is the primary activity in the Write Back (WB) stage of a pipeline?**
A: Writing the result of the instruction (from the ALU or memory) back into the register file.
47. **Q: In a five-stage pipeline, how many instructions can be in execution during a single clock cycle?**
A: Up to five instructions.
48. **Q: What is the general direction of data and instruction flow through the pipelined datapath stages?**
A: From left to right, from the IF stage to the WB stage.
49. **Q: What is the first exception to the left-to-right data flow in a pipelined datapath?**
A: The write-back stage, which sends the result back to the register file located in the middle of the datapath.
50. **Q: What is the second exception to the left-to-right data flow in a pipelined datapath?**
A: The selection of the next PC value, which can choose a branch address from the MEM stage.
51. **Q: The right-to-left data flow from the write-back stage can lead to what type of hazard?**
A: Data hazards.
52. **Q: The right-to-left data flow for PC selection (branching) can lead to what type of hazard?**
A: Control hazards.
53. **Q: What hardware components are added between the stages of a datapath to implement pipelining?**
A: Pipeline registers.
54. **Q: The pipeline register located between the Instruction Fetch and Instruction Decode stages is named ____.**
A: IF/ID.

55. **Q: What is the purpose of pipeline registers in a pipelined datapath?**
A: To hold the data and control information for an instruction as it moves from one stage to the next, allowing stages to be shared by different instructions simultaneously.
56. **Q: In the IF stage of a 'ld' instruction, what happens to the PC value?**
A: It is incremented by 4 and written back into the PC for the next cycle, and a copy is saved in the IF/ID pipeline register.
57. **Q: In the ID stage of a 'ld' instruction, what values are stored in the ID/EX pipeline register?**
A: The values from two read registers, the sign-extended immediate value, and the PC address.
58. **Q: What calculation is performed in the EX stage of a 'ld' instruction?**
A: The ALU adds the contents of a register to the sign-extended immediate value to calculate the memory address.
59. **Q: What happens in the MEM stage of a 'ld' instruction?**
A: The data memory is read using the address from the EX/MEM pipeline register, and the loaded data is placed in the MEM/WB pipeline register.
60. **Q: In the WB stage of a 'ld' instruction, data is read from the _____ pipeline register and written into the register file.**
A: MEM/WB
61. **Q: How does a 'store' instruction make the data to be stored available during the MEM stage?**
A: The data is read from the register file in the ID stage and passed through the ID/EX and EX/MEM pipeline registers.
62. **Q: What happens in the Write-Back (WB) stage for a 'store' instruction?**
A: Nothing happens in the WB stage for a store instruction, as the result was already written to memory in the MEM stage.
63. **Q: To avoid structural hazards, each logical component of the datapath, such as the ALU, can be used only within a single _____.**
A: pipeline stage
64. **Q: What was the original design bug in handling the 'ld' instruction's write-back stage?**
A: The write register number was being supplied by the instruction in the IF/ID pipeline register, not the 'ld' instruction that was actually in the WB stage.
65. **Q: How was the 'ld' instruction bug fixed in the pipelined datapath design?**
A: The destination register number is passed from the ID stage through the ID/EX, EX/MEM, and MEM/WB pipeline registers to be used in the WB stage.
66. **Q: What type of pipeline diagram shows the complete execution of multiple instructions over time, with clock cycles on one axis and instructions on the other?**
A: A multiple-clock-cycle pipeline diagram.
67. **Q: What type of pipeline diagram shows the state of the entire datapath during a single clock cycle?**
A: A single-clock-cycle pipeline diagram.
68. **Q: A single-clock-cycle diagram represents a vertical slice through what other type of diagram?**
A: A multiple-clock-cycle diagram.
69. **Q: How are control signals for the EX, MEM, and WB stages passed down the pipeline?**
A: They are generated in the ID stage and then stored in the control portions of the pipeline registers (ID/EX, EX/MEM, MEM/WB).
70. **Q: Which two control signals are set during the Execution/address calculation (EX) stage?**
A: ALUOp and ALUSrc.
71. **Q: Which three control signals are set during the Memory access (MEM) stage?**
A: Branch, MemRead, and MemWrite.

72. **Q: Which two control signals are set during the Write-back (WB) stage?**
A: MemtoReg and RegWrite.
73. **Q: Why do the IF and ID stages not require special control signals to be passed to them?**
A: Their primary operations (reading instruction memory, writing the PC, reading registers) are always asserted or are determined by fixed fields in the instruction format.
74. **Q: In the corrected datapath, the write register number for the WB stage comes from which pipeline register?**
A: The MEM/WB pipeline register.
75. **Q: What information must be stored in the IF/ID pipeline register for a 'beq' instruction to potentially work later on?**
A: The incremented PC address.
76. **Q: During the EX stage of a 'store' instruction, what two values are placed into the EX/MEM pipeline register?**
A: The calculated effective address and the second register value (the data to be stored).
77. **Q: According to the 'Check Yourself' section, is it true that allowing some instructions to take fewer stages always increases pipeline performance?**
A: No, because pipeline throughput is determined by the clock cycle time, not the latency of individual instructions.
78. **Q: The number of pipeline stages per instruction primarily affects an instruction's _____, not the pipeline's throughput.**
A: latency
79. **Q: According to the 'Check Yourself' section, what is a potential benefit of making a pipeline longer (more stages)?**
A: A longer pipeline can allow for a shorter clock cycle time, which could improve overall performance (throughput).
80. **Q: The IF/ID pipeline register must be wide enough to hold both the 32-bit instruction and the incremented ____ address.**
A: 64-bit PC
81. **Q: The control logic generates control signals for the EX, MEM, and WB stages during which pipeline stage?**
A: The Instruction Decode (ID) stage.
82. **Q: The control line _____ determines whether the ALU gets its second operand from the register file or the sign-extended immediate field.**
A: ALUSrc
83. **Q: The control line _____ determines whether the value written to the register file comes from the ALU result or from data memory.**
A: MemtoReg
84. **Q: The _____ control signal must be asserted to enable writing to the register file in the WB stage.**
A: RegWrite
85. **Q: What two signals must both be asserted for the 'PCSrc' multiplexor to select the branch target address?**
A: The 'Branch' signal (from control) and the 'Zero' signal (from the ALU).

14 Introduction

“What do you mean, why’s it got to be built? It’s a bypass. You’ve got to build bypasses.”
 — Douglas Adams, *The Hitchhiker’s Guide to the Galaxy*, 1979

The examples in the previous section showed the power of pipelined execution with **independent** instructions. Now it’s time to take off the rose-colored glasses and look at what happens with **real programs** that have **data dependences**.

14.1 A Sequence with Many Dependences

Consider this RISC-V code sequence where the last four instructions all depend on the result of the first instruction (x2):

```
sub  x2, x1, x3    // Register x2 written by sub
and  x12, x2, x5   // 1st operand (x2) depends on sub
or   x13, x6, x2   // 2nd operand (x2) depends on sub
add  x14, x2, x2   // 1st & 2nd (x2) depend on sub
sd   x15, 100(x2)  // Base (x2) depends on sub
```

If register x2 had the value **10** before the **sub** instruction and **−20** afterwards, the programmer intends that **−20** will be used in all following instructions that refer to x2.

15 Visualizing Pipeline Data Hazards (Figure 4.50)

The diagram below shows how the five instructions execute over time. Register x2 is **written in clock cycle 5** (WB stage of **sub**), so the proper value is unavailable before CC5.

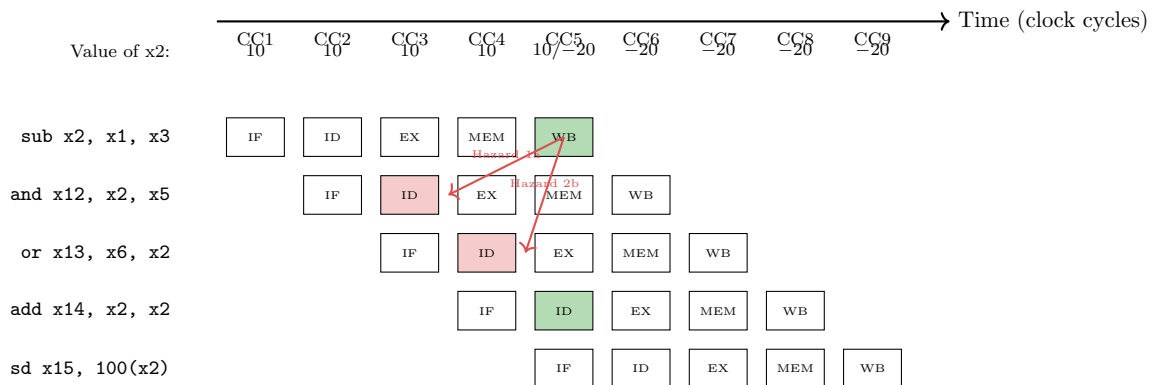


Figure 8: **Figure 4.50:** Pipelined dependences. The **and** and **or** instructions read x2 *before* it is written (red arrows go backward in time ⇒ **data hazards**).

Key Insight

- **and** reads x2 in CC3 (ID stage), but **sub** writes x2 in CC5 (WB) ⇒ **Hazard 1a**.
- **or** reads x2 in CC4 (ID stage), but **sub** writes x2 in CC5 (WB) ⇒ **Hazard 2b**.
- **add** reads x2 in CC5 (ID stage) ⇒ **No hazard** (register file internal forwarding).
- **sd** reads x2 in CC6 ⇒ **No hazard**.

16 Forwarding (Bypassing)

The desired result is available at the **end of the EX stage** of the **sub** instruction (CC3). The **and** and **or** instructions need the data at the **beginning of their EX stage** (CC4 and CC5 respectively).

Solution: We can “forward” the data as soon as it is available to any units that need it, *before* it is written to the register file.

16.1 Hazard Conditions

A notation names the fields of the pipeline registers:

- `ID/EX.RegisterRs1`: The first source register number in the ID/EX pipeline register.
- `EX/MEM.RegisterRd`: The destination register number in the EX/MEM pipeline register.

The two pairs of hazard conditions are:

Type	Condition
1a	<code>EX/MEM.RegisterRd = ID/EX.RegisterRs1</code>
1b	<code>EX/MEM.RegisterRd = ID/EX.RegisterRs2</code>
2a	<code>MEM/WB.RegisterRd = ID/EX.RegisterRs1</code>
2b	<code>MEM/WB.RegisterRd = ID/EX.RegisterRs2</code>

Example: Classifying the Dependences

In the sequence from above:

- `sub-and` is a **Type 1a** hazard: `EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x2`.
- `sub-or` is a **Type 2b** hazard: `MEM/WB.RegisterRd = ID/EX.RegisterRs2 = x2`.
- `sub-add`: **Not a hazard** (register file supplies data during ID stage of `add`).
- `sub-sd`: **Not a hazard** (`sd` reads `x2` the cycle *after* `sub` writes it).

16.2 Avoiding Incorrect Forwarding

Some instructions do not write registers. We must check:

1. The `RegWrite` signal is active.
2. The destination register is **not** `x0` (since `x0` must always be 0).

17 Forwarding Hardware (Figure 4.52)

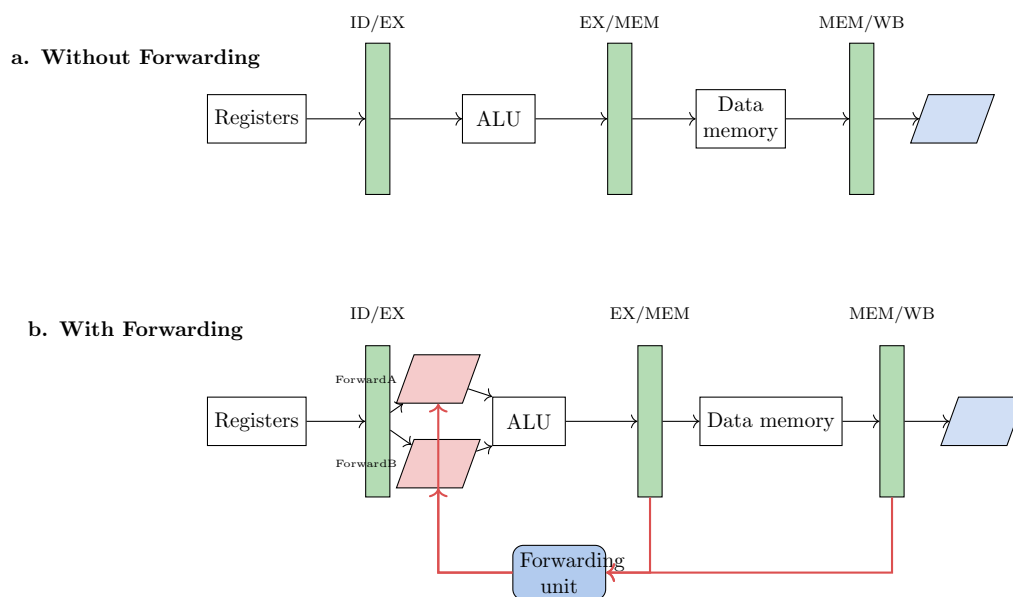


Figure 9: **Figure 4.52:** (a) ALU and pipeline registers *without* forwarding. (b) With forwarding: multiplexers added to ALU inputs, controlled by the Forwarding Unit.

18 Forwarding Control Values (Figure 4.53)

Mux Control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Table 1: **Figure 4.53:** Control values for the forwarding multiplexors.

18.1 Forwarding Logic Conditions

EX Hazard (Forward from EX/MEM)

```

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 10

if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 10

```

MEM Hazard (Forward from MEM/WB)

```

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 01

if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 01

```

18.2 Double Data Hazard

Consider:

```

add x1, x1, x2
add x1, x1, x3
add x1, x1, x4

```

Both the first and second instructions write to **x1** before the third instruction reads it. The result should be forwarded from the **MEM stage** (most recent). The MEM hazard condition must check that there is **NOT** a more recent EX hazard.

Revised MEM Hazard Logic (Double Data Hazard)

The key insight: only forward from MEM/WB if there is **NOT** a more recent EX hazard.

```
// MEM hazard for Rs1 (with double hazard check)
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and NOT(EX/MEM.RegWrite
            and (EX/MEM.RegisterRd != 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 01
```

```
// MEM hazard for Rs2 (with double hazard check)
if (MEM/WB.RegWrite
    and (MEM/WB.RegisterRd != 0)
    and NOT(EX/MEM.RegWrite
            and (EX/MEM.RegisterRd != 0)
            and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    and (MEM/WB.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 01
```

The EX hazard conditions remain unchanged:

```
// EX hazard (always takes priority - most recent value)
if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
    ForwardA = 10
```

```
if (EX/MEM.RegWrite
    and (EX/MEM.RegisterRd != 0)
    and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
    ForwardB = 10
```

19 Data Hazards and Stalls

Load-Use Hazard: Forwarding Cannot Help!

When a load instruction is immediately followed by an instruction that uses its result, forwarding **cannot** save the day. The data is still being read from memory in CC4 (MEM stage) while the dependent instruction needs it at the *beginning* of CC4 (EX stage).

19.1 Hazard Detection Unit

The hazard detection unit operates during the **ID stage** to insert a stall:

Load-Use Hazard Detection

```
if (ID/EX.MemRead
    and ((ID/EX.RegisterRd = IF/ID.RegisterRs1)
        or (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
    stall the pipeline
```

19.2 How Stalls Work

When stalling:

1. **Prevent PC** from updating (re-fetch the same instruction).
2. **Prevent IF/ID register** from changing (re-decode the same instruction).

3. Insert a bubble (NOP) by setting all control signals in ID/EX to 0.

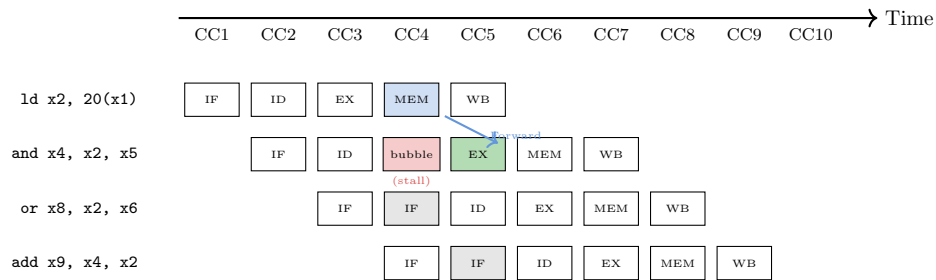


Figure 10: **Figure 4.57:** How stalls are inserted. The **and** instruction (dependent on **ld**) gets a **bubble** inserted at CC4. The PC and IF/ID registers are frozen, causing subsequent instructions to repeat their IF stage. After the 1-cycle stall, forwarding from MEM/WB to EX handles the dependence.

20 The Big Picture

Compiler's Role

Although the **hardware** generally resolves hazards to ensure correct execution, the **compiler** must understand the pipeline to achieve the best performance. Otherwise, unexpected stalls will reduce the performance of the compiled code. Compilers can reorder instructions to avoid hazards.

21 Flashcards

1. **Q: What is a ‘data hazard’ in the context of pipelined execution?**
A: It is an obstacle to pipelined execution where an instruction depends on the result of a previous instruction that is still in the pipeline.
2. **Q: In a pipeline diagram, how do data hazards visually manifest themselves?**
A: They appear as dependence lines that must go backward in time from when data is written to when it is read.
3. **Q: What is the primary hardware technique used to resolve most data hazards without stalling the pipeline?**
A: Forwarding, also known as bypassing, which passes the result directly from where it is computed to where it is needed.
4. **Q: For an ALU operation in the EX stage, the result of a previous instruction is needed at the _____ of the EX stage.**
A: beginning
5. **Q: How is a data hazard resolved if a register is read and written in the same clock cycle?**
A: The register file hardware is designed so the read gets the value that was written in the first half of the cycle, effectively performing an internal forward.
6. **Q: What does the notation ‘ID/EX.RegisterRs1’ specifically refer to in pipeline hazard detection?**
A: It refers to the number of the first source register (Rs1) for the instruction currently in the EX stage, held in the ID/EX pipeline register.
7. **Q: What is the first condition of a type ‘1a’ data hazard, also known as an EX hazard?**
A: The condition is $\text{EX/MEM.RegisterRd} = \text{ID/EX.RegisterRs1}$, where the previous instruction’s destination matches the current instruction’s first source.
8. **Q: What is the first condition of a type ‘2a’ data hazard, also known as a MEM hazard?**
A: The condition is $\text{MEM/WB.RegisterRd} = \text{ID/EX.RegisterRs1}$, where the destination of the instruction in the MEM stage matches the current instruction’s first source.
9. **Q: In the sequence ‘sub x2, x1, x3’ followed immediately by ‘and x12, x2, x5’, what specific type of hazard is detected?**
A: This is a type 1a hazard, as the and instruction needs the result of sub from the EX/MEM pipeline register.
10. **Q: In the sequence where ‘sub x2, x1, x3’ is followed two instructions later by ‘or x13, x6, x2’, what specific type of hazard is detected?**
A: This is a type 2b hazard, as the or instruction needs the result of sub from the MEM/WB pipeline register.
11. **Q: Why does the sequence ‘sub x2, ...’ followed by ‘add x14, x2, x2’ not present a data hazard that requires forwarding?**
A: Because the register file supplies the proper data during the ID stage of add, as sub will have completed its WB stage in the previous cycle.
12. **Q: What control signal must be checked in the forwarding logic to ensure that an instruction actually writes to a register?**
A: The RegWrite signal for the instruction in the EX or MEM stage must be checked.
13. **Q: Why must forwarding logic include the condition $\text{RegisterRd} \neq 0$?**
A: To prevent forwarding the result of an instruction that writes to register x0, which must always be treated as the value zero.
14. **Q: What hardware is added to the datapath to implement forwarding to the ALU?**
A: Multiplexors are added to each of the ALU inputs to select data from a register file read port or a pipeline register.
15. **Q: The forwarding unit, which generates control signals for the ALU input multiplexors, is located in which pipeline stage?**
A: The EX stage.

16. Q: What does the forwarding control signal ForwardA = 10 signify?

A: It instructs the ALU's 'A' input multiplexor to select the result from the prior instruction (via the EX/MEM pipeline register).

17. Q: What does the forwarding control signal ForwardB = 01 signify?

A: It instructs the ALU's 'B' input multiplexor to select the result from the MEM stage (via the MEM/WB pipeline register).

18. Q: In a sequence like 'add x1, x1, x2' followed by 'add x1, x1, x3', which result should be forwarded to the second add?

A: The result from the most recent instruction, which would be the first add instruction currently in the MEM stage.

19. Q: What check must be added to the MEM hazard detection logic to correctly handle back-to-back dependent instructions?

A: It must check that there is NOT a more recent EX hazard for the same register, ensuring the newest data is always used.

20. Q: What is the one specific case where forwarding cannot prevent a pipeline stall?

A: A load-use hazard, which occurs when an instruction tries to read a register immediately following a load instruction that writes to that same register.

21. Q: Why is forwarding insufficient to resolve a load-use data hazard?

A: The data from the load is not available until the end of the MEM stage (clock cycle 4), but the dependent instruction needs it at the beginning of its EX stage (also clock cycle 4).

22. Q: A _____ is an instruction that performs no operation and changes no state, used to resolve a load-use hazard.

A: nop (no operation)

23. Q: What is the function of the 'hazard detection unit' in a pipelined datapath?

A: It detects load-use hazards and stalls the pipeline to prevent incorrect execution.

24. Q: The hazard detection unit operates during the _____ stage to insert a stall between the load and the dependent instruction.

A: ID

25. Q: What is the precise condition the hazard detection unit checks for to initiate a stall?

A: if (ID/EX.MemRead and ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or (ID/EX.RegisterRd = IF/ID.RegisterRs2)))

26. Q: When the hazard detection unit stalls the pipeline, which two registers are prevented from being updated?

A: The PC (Program Counter) and the IF/ID pipeline register.

27. Q: How is a nop or 'bubble' effectively inserted into the pipeline during a stall?

A: By deasserting all control signals (setting them to 0) in the EX, MEM, and WB control fields of the ID/EX pipeline register.

28. Q: What entity, other than the hardware itself, must understand the pipeline to achieve the best performance by avoiding stalls?

A: The compiler.

29. Q: The hazard detection unit controls a multiplexor that chooses between the real control values from the ID stage and a set of _____ to create a bubble.

A: all 0s

30. Q: To accommodate immediate values for instructions like addi, a 2:1 multiplexor is added to choose between the ForwardB multiplexor output and the _____.

A: signed immediate

31. Q: In a five-stage pipeline, if a load instruction is in the MEM stage (cycle 4), a dependent instruction would be in the _____ stage, creating a conflict.

A: EX

32. Q: Term: Stall

A: Definition: A technique to resolve a hazard by pausing the pipeline for one or more clock cycles.

33. **Q: The full logic for an EX hazard on the first ALU operand (Rs1) is: if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0) and (EX/MEM.RegisterRd = ____)) ForwardA = 10.**
 A: ID/EX.RegisterRs1
34. **Q: The full logic for a MEM hazard on the second ALU operand (Rs2) is: if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0) and (MEM/WB.RegisterRd = ____)) ForwardB = 01.**
 A: ID/EX.RegisterRs2
35. **Q: What is the effect of stalling on instructions that are in the fetch and decode stages?**
 A: They are prevented from making progress; the instruction in IF is re-fetched and the instruction in ID is re-decoded in the next cycle.
36. **Q: Forwarding can resolve a data hazard between a store instruction's data source and a preceding ALU instruction because the data can be forwarded from the ____ or ____ stages to the MEM stage.**
 A: EX, MEM
37. **Q: What is a data hazard in the context of pipelined execution?**
 A: It is an obstacle where an instruction depends on the result of a previous instruction that has not yet completed its execution in the pipeline.
38. **Q: In a five-stage pipeline without forwarding, when is the result of an instruction written to the register file?**
 A: During the middle of the fifth clock cycle, in the Write Back (WB) stage.
39. **Q: What problem arises if an instruction reads a register before a preceding instruction has written its new value back to it?**
 A: The instruction will read the old, incorrect value from the register, leading to incorrect program execution.
40. **Q: The problem of a data hazard becomes apparent in a pipeline diagram when a dependence line goes ____ in time.**
 A: backward
41. **Q: How does register file hardware design resolve a potential hazard when a register is read and written in the same clock cycle?**
 A: It assumes the write occurs in the first half of the cycle and the read occurs in the second half, so the read gets the newly written value.
42. **Q: At what point in the pipeline is the result of an ALU operation, like sub, first available?**
 A: At the end of the EX (Execute) stage.
43. **Q: At what point in the pipeline does an instruction like and or or need its operand values?**
 A: At the beginning of its EX (Execute) stage.
44. **Q: What is the general principle of forwarding (or bypassing)?**
 A: It is the process of taking a result directly from a pipeline register of a preceding instruction and sending it to a functional unit (like the ALU) for a subsequent instruction, bypassing the register file.
45. **Q: What does the notation ID/EX.RegisterRs1 refer to in pipeline analysis?**
 A: It refers to the field in the ID/EX pipeline register that holds the number of the first source register (rs1).
46. **Q: What is the first condition for detecting a type 1a data hazard?**
 A: The condition is EX/MEM.RegisterRd = ID/EX.RegisterRs1.
47. **Q: What is the first condition for detecting a type 2a data hazard?**
 A: The condition is MEM/WB.RegisterRd = ID/EX.RegisterRs1.
48. **Q: In the sequence sub x2, x1, x3 followed by and x12, x2, x5, what type of hazard exists between them?**
 A: A type 1a hazard, where EX/MEM.RegisterRd = ID/EX.RegisterRs1 = x2.
49. **Q: In the sequence sub x2, x1, x3 followed two instructions later by or x13, x6, x2, what type of hazard exists?**
 A: A type 2b hazard, where MEM/WB.RegisterRd = ID/EX.RegisterRs2 = x2.

50. **Q: Why is there no data hazard between sub x2,... and add x14, x2, x2 if the add is three instructions later?**
A: Because the register file supplies the proper data during the ID stage of add, as the sub instruction will have completed its WB stage.
51. **Q: What additional check is necessary to prevent incorrect forwarding from instructions that do not write to a register?**
A: The hardware must check if the RegWrite control signal for the instruction in the EX or MEM stage is active (asserted).
52. **Q: Why must the hardware specifically prevent forwarding the result of an instruction whose destination is register x0?**
A: Because register x0 must always read as zero, and forwarding a non-zero result would violate this architectural rule.
53. **Q: What is the role of the forwarding unit?**
A: The forwarding unit contains the logic to detect data hazards and control the ALU input multiplexors to select the correct forwarded data.
54. **Q: In which pipeline stage does the forwarding control logic operate?**
A: It operates in the EX stage, as this is where the ALU forwarding multiplexors are located.
55. **Q: An EX hazard is a hazard between the instruction in the EX stage and the result of the instruction currently in the ____ stage.**
A: MEM
56. **Q: What control signal values are set for ForwardA if an EX hazard is detected for the first ALU operand (rs1)?**
A: The control value for ForwardA is set to 10.
57. **Q: A MEM hazard is a hazard between the instruction in the EX stage and the result of the instruction currently in the ____ stage.**
A: WB
58. **Q: What control signal values are set for ForwardB if a MEM hazard is detected for the second ALU operand (rs2)?**
A: The control value for ForwardB is set to 01.
59. **Q: If an operand has a data hazard from both the MEM stage and the WB stage, which result should be forwarded?**
A: The result from the MEM stage should be forwarded, as it is from the more recent instruction.
60. **Q: What is a load-use hazard?**
A: A situation where an instruction tries to read a register immediately following a ld (load) instruction that writes to that same register.
61. **Q: Why can't forwarding alone resolve a load-use data hazard?**
A: Because the data from the ld instruction is not available from memory until the end of the MEM stage, but the dependent instruction needs it at the beginning of its EX stage one cycle earlier.
62. **Q: What hardware unit is required specifically to handle load-use hazards?**
A: A hazard detection unit.
63. **Q: In which pipeline stage must the hazard detection unit operate to stall the pipeline for a load-use hazard?**
A: It must operate during the ID (Instruction Decode) stage.
64. **Q: What is the condition checked by the hazard detection unit to initiate a stall?**
A: if (ID/EX.MemRead and ((ID/EX.RegisterRd = IF/ID.RegisterRs1) or (ID/EX.RegisterRd = IF/ID.RegisterRs2)))
65. **Q: What is the primary action taken by the hardware when a pipeline stall is initiated?**
A: It prevents the PC register and the IF/ID pipeline register from changing for one clock cycle.
66. **Q: What is a 'nop' instruction in the context of pipelining?**
A: A nop (or bubble) is an instruction that does no operation to change state, effectively inserted into the pipeline to create a delay.

67. **Q: How is a nop or bubble created and inserted into the pipeline by the hazard detection unit?**
A: By deasserting all the control fields (setting them to 0) in the ID/EX pipeline register for the stalled instruction.
68. **Q: What is the effect of a stall on the instruction in the ID stage and the instruction in the IF stage?**
A: Both instructions are delayed; the ID stage instruction repeats decoding, and the IF stage instruction is re-fetched from the same PC value.
69. **Q: While the compiler usually relies on hardware to resolve hazards, how can a compiler improve performance?**
A: The compiler can reorder instructions to avoid sequences that would cause stalls, thus improving performance.
70. **Q: What is the purpose of the ID/EX.MemRead signal in the load-use hazard detection logic?**
A: It identifies that the instruction currently in the EX stage is a load, which is the only instruction that can cause this type of stall.
71. **Q: What does a control value of 00 for the ForwardA or ForwardB multiplexor signify?**
A: It signifies that no forwarding is needed for that input, and the value should come from the register file via the ID/EX pipeline register.

22 Introduction

“To make a computer with automatic program-interruption facilities behave [sequentially] was not an easy matter, because the number of instructions in various stages of processing when an interrupt signal occurs may be large.”

— Fred Brooks, Jr., *Planning a Computer System: Project Stretch*, 1962

Thus far, we have limited our concern to hazards involving arithmetic operations and data transfers. However, there are also pipeline hazards involving **conditional branches**.

Control Hazard (Branch Hazard)

A **control hazard** is a delay in determining the proper instruction to fetch, caused by a conditional branch instruction. An instruction must be fetched at every clock cycle to sustain the pipeline, yet in our basic design the decision about whether to branch doesn't occur until the **MEM pipeline stage**.

22.1 Why Control Hazards are Different

This section on control hazards is shorter than the previous sections on data hazards for three reasons:

1. Control hazards are **relatively simple** to understand.
2. They occur **less frequently** than data hazards.
3. There is **nothing as effective** against control hazards as forwarding is against data hazards.

Hence, we use **simpler schemes**. We look at two schemes for resolving control hazards and one optimization.

23 The Problem: Branch Delay

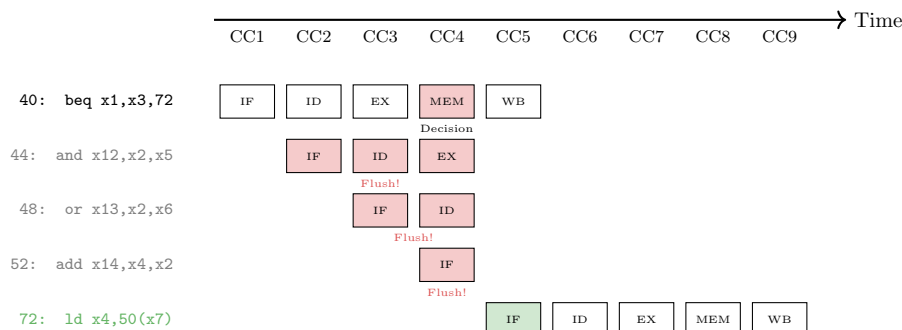


Figure 11: **Figure 4.59:** The impact of the pipeline on a branch instruction. Without optimization, the branch decision is made in the MEM stage (CC4), so **three sequential instructions** are fetched and must be flushed if the branch is taken.

The Problem

Without intervention, three following instructions will begin execution **before** the **beq** instruction determines whether to branch to location 72.

24 Scheme 1: Assume Branch Not Taken

Stalling until the branch is complete is too slow. A better improvement is to **predict that the conditional branch will not be taken** and continue execution down the sequential instruction stream.

- If the branch is **not taken**: Prediction correct, no penalty.
- If the branch is **taken**: The instructions that were fetched and decoded must be **discarded** (flushed), and execution continues at the branch target.

24.1 Flushing Instructions

Flush

To **flush** instructions means to discard them from the pipeline, usually due to an unexpected event like a mispredicted branch.

To discard instructions, we change the original control values to **0s**, similar to what we did for load-use data hazards. The difference is that we must change the instructions in **IF, ID, and EX stages** when the branch reaches the MEM stage.

25 Reducing the Delay of Branches

One way to improve conditional branch performance is to reduce the cost of a taken branch. If we move the conditional branch execution **earlier in the pipeline**, fewer instructions need to be flushed.

25.1 Moving Branch Decision to ID Stage

Moving the branch decision up requires two actions:

1. **Computing the branch target address:** Move the branch adder from EX stage to ID stage. We already have the PC and immediate field in the IF/ID pipeline register.
2. **Evaluating the branch decision:** For `beq`, compare two register reads during ID using an **equality test unit** (XOR bits, then OR the results).

25.2 Complications

Moving the branch test to ID introduces new hardware requirements:

New Forwarding and Hazard Detection

1. **Forwarding to ID:** The equality test unit may need results from EX/MEM or MEM/WB pipeline registers. New forwarding logic is required.
2. **New Stall Conditions:**
 - ALU instruction immediately before branch → **1-cycle stall**
 - Load instruction immediately before branch → **2-cycle stall**

25.3 The IF.Flush Control Line

To flush instructions in the IF stage, we add a control line called `IF.Flush` that **zeros the instruction field** of the IF/ID pipeline register. This transforms the fetched instruction into a **nop** (no operation).

Example: Pipelined Branch (Optimized)

Consider this instruction sequence with branch execution moved to ID stage:

```

36 sub  x10, x4, x8
40 beq  x1, x3, 16    // Branch to 40+16*2=72
44 and  x12, x2, x5
48 or   x13, x2, x6
...
72 ld   x4, 50(x7)    // Branch target

```

Result: If branch is taken, only **one instruction** (the one being fetched at CC3) needs to be flushed, instead of three.

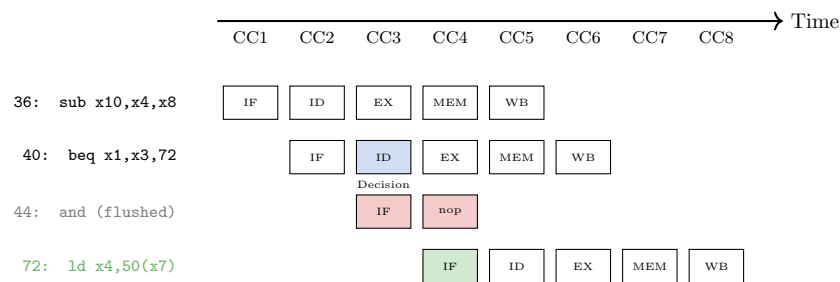


Figure 12: **Figure 4.60:** With branch execution moved to ID stage, only **one bubble** is needed on a taken branch instead of three.

26 Dynamic Branch Prediction

Assuming a conditional branch is not taken is a simple form of **static** branch prediction. With deeper pipelines and multiple issue, a simple static prediction scheme wastes too much performance. With more hardware, it is possible to **predict branch behavior during program execution**.

Dynamic Branch Prediction

Prediction of branches at runtime using runtime information. The approach looks up the address of the instruction to see if the branch was taken the last time it was executed.

26.1 Branch Prediction Buffer (Branch History Table)

Branch Prediction Buffer

A **branch prediction buffer** (or **branch history table**) is a small memory indexed by the lower portion of the branch instruction's address. It contains one or more bits indicating whether the branch was recently taken or not.

Note: The prediction is just a **hint**. We don't even know if the bit was set by this branch or another branch with the same low-order address bits. If the hint is wrong:

1. Incorrectly predicted instructions are deleted.
2. The prediction bit is **inverted** and stored back.
3. The proper sequence is fetched and executed.

26.2 The Problem with 1-Bit Prediction

1-Bit Prediction Shortcoming

Even if a branch is almost always taken (like a loop), a 1-bit scheme can mispredict **twice** per loop execution rather than once:

- On the **first iteration**: The bit was flipped on the previous loop exit.
- On the **last iteration**: The branch is not taken, but the bit predicts taken.

Example: Loop Prediction Accuracy

Consider a loop branch that branches 9 times in a row, then is not taken once.

With 1-bit prediction:

- Mispredicts iteration 1 (bit was flipped on last exit)
- Mispredicts iteration 10 (exit iteration)
- Accuracy: $\frac{8}{10} = 80\%$

Ideally: Accuracy should match the 90% taken frequency.

26.3 2-Bit Branch Prediction

To remedy the 1-bit weakness, **2-bit prediction schemes** are often used. A prediction must be wrong **twice** before it is changed.

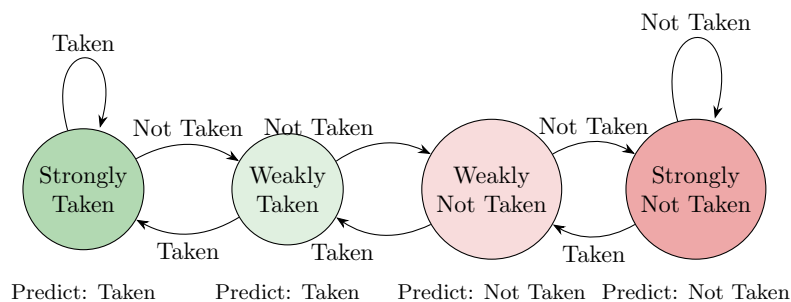


Figure 13: **Figure 4.61:** The states in a 2-bit prediction scheme. A prediction must be wrong twice before it changes. This is a **counter-based predictor**: incremented on accurate prediction, decremented otherwise.

Result: A branch that strongly favors taken or not taken will be mispredicted **only once** when its pattern is briefly interrupted.

27 Advanced Prediction Techniques

27.1 Branch Target Buffer

A branch predictor tells us *whether* a branch is taken, but still requires calculation of the branch target. A **branch target buffer** caches the destination PC or instruction.

Branch Target Buffer

A structure that caches the destination PC or destination instruction for a branch. It is usually organized as a **cache with tags**, making it more costly than a simple prediction buffer.

27.2 Correlating Predictor

The 2-bit scheme uses only local information about a particular branch. A **correlating predictor** combines:

- **Local behavior:** How this specific branch has behaved.
- **Global information:** How recently executed branches have behaved.

27.3 Tournament Branch Predictor

A **tournament predictor** uses multiple prediction mechanisms and a selector that chooses which predictor to use for each branch based on which has been more accurate.

28 Reducing Branches: Conditional Move

One way to reduce the number of conditional branches is to add **conditional move instructions**. Instead of changing the PC with a conditional branch, the instruction conditionally changes the destination register.

ARMv8 CSEL Instruction

CSEL X8, X11, X4, NE

- If condition NE (not equal) is true: X8 = X11
- If condition is false: X8 = X4

This avoids a branch entirely!

29 Check Yourself

Consider three branch prediction schemes with a 2-cycle misprediction penalty and 0-cycle correct prediction penalty:

- **Predict Not Taken**
- **Predict Taken**
- **Dynamic Predictor** (90% accurate)

Taken %	Predict NT	Predict T	Dynamic 90%	Best Choice
5%	95% correct	5% correct	90% correct	Predict Not Taken
95%	5% correct	95% correct	90% correct	Predict Taken
70%	30% correct	70% correct	90% correct	Dynamic

Table 2: Choosing the best branch prediction scheme based on taken frequency.

30 Flashcards

1. **Q: What is a control hazard in a processor pipeline?**
A: It is a delay in determining the proper instruction to fetch, caused by a conditional branch instruction.
2. **Q: In the non-optimized five-stage pipeline described, in which stage is the decision about whether to branch made?**
A: The decision is made in the MEM (Memory Access) pipeline stage.
3. **Q: Without any optimization, how many sequential instructions following a branch are fetched before the branch decision is made in the MEM stage?**
A: Three sequential instructions are fetched and begin execution.
4. **Q: The technique of discarding instructions in a pipeline, usually due to an unexpected event like a mispredicted branch, is called ____.**
A: flushing
5. **Q: What is the simplest static branch prediction scheme mentioned in the text?**
A: The 'Assume Branch Not Taken' scheme, where execution continues down the sequential instruction stream.
6. **Q: In the 'Assume Branch Not Taken' scheme, what happens if the conditional branch is actually taken?**
A: The instructions that were being fetched and decoded must be discarded, and execution continues at the branch target.
7. **Q: How are instructions discarded or flushed when a branch is mispredicted in the 'Assume Not Taken' scheme?**
A: The original control values for the instructions in the IF, ID, and EX stages are changed to 0s.
8. **Q: What is the primary method discussed to improve conditional branch performance by reducing the cost of a taken branch?**
A: Moving the conditional branch execution earlier in the pipeline, from the MEM stage to the ID stage.
9. **Q: What are the two actions that must occur earlier to move conditional branch execution to the ID stage?**
A: Computing the branch target address and evaluating the branch decision must both occur earlier.
10. **Q: What hardware modification is needed in the ID stage to calculate the branch target address earlier?**
A: The branch adder is moved from the EX stage to the ID stage.
11. **Q: What hardware modification is needed in the ID stage to evaluate the branch decision for 'branch if equal' earlier?**
A: An equality test unit is added to compare two register values during the ID stage.
12. **Q: When branch execution is moved to the ID stage, what new hardware complication arises regarding data dependencies?**
A: New forwarding logic is required, as the equality test unit in ID may need results from later pipeline stages (EX/MEM or MEM/WB).
13. **Q: If an ALU instruction immediately precedes a branch that depends on its result, what is required when branch execution is in the ID stage?**
A: A stall will be required because the ALU result is produced in the EX stage, after the branch's ID cycle.
14. **Q: If a load instruction is immediately followed by a conditional branch that depends on its result, how many stall cycles are needed if branch execution is in the ID stage?**
A: Two stall cycles are needed, as the load result is not available until the end of the MEM cycle.
15. **Q: By moving branch execution to the ID stage, the penalty of a taken branch is reduced to how many instructions?**
A: The penalty is reduced to only one instruction, which is the one currently being fetched.
16. **Q: What is the purpose of the 'IF.Flush' control line?**
A: It zeros the instruction field of the IF/ID pipeline register, effectively transforming the fetched instruction into a nop.

17. **Q: Prediction of branches at runtime using runtime information is known as ____.**
A: dynamic branch prediction
18. **Q: What is a branch prediction buffer, also known as a branch history table?**
A: It is a small memory indexed by the lower portion of a branch instruction's address, containing bits that indicate if the branch was recently taken.
19. **Q: What is the primary performance shortcoming of a simple 1-bit dynamic branch prediction scheme?**
A: If a branch is almost always taken (like in a loop), it will be mispredicted twice for each full loop execution rather than just once.
20. **Q: For a loop branch that is taken nine times and not taken once, what is the prediction accuracy using a 1-bit scheme?**
A: The accuracy is 80%, with two incorrect predictions (on the first and last iterations) and eight correct ones.
21. **Q: Why does a 1-bit predictor mispredict the first iteration of a loop that was just completed?**
A: The prediction bit was flipped to 'not taken' on the final, exiting iteration of the previous loop execution.
22. **Q: How does a 2-bit branch prediction scheme improve over a 1-bit scheme?**
A: A prediction must be wrong twice before the prediction state is changed, leading to only one misprediction for highly regular loops.
23. **Q: In a 2-bit counter-based predictor, what action is taken when the prediction is accurate?**
A: The counter is incremented (or moved towards a more strongly-taken/not-taken state).
24. **Q: In a 2-bit counter-based predictor, what action is taken when the prediction is inaccurate?**
A: The counter is decremented (or moved towards the weakly-taken/not-taken state, and eventually flips).
25. **Q: What structure caches the destination PC or destination instruction for a branch to reduce the taken-branch penalty?**
A: A branch target buffer.
26. **Q: How does a branch target buffer differ from a simpler branch prediction buffer?**
A: It is usually organized as a cache with tags, making it more costly, and it stores the target address or instruction, not just a prediction bit.
27. **Q: What is a correlating predictor?**
A: A branch predictor that combines the local behavior of a branch with global information about recently executed branches.
28. **Q: What is a tournament branch predictor?**
A: A predictor that uses multiple prediction schemes and a selection mechanism to choose the best one for a given branch.
29. **Q: What type of instruction can be added to an instruction set to reduce the number of conditional branches?**
A: Conditional move instructions.
30. **Q: What does the ARMv8 'CSEL' instruction do?**
A: It conditionally selects one of two source registers to copy to a destination register based on a condition code.
31. **Q: For a branch with 5% taken frequency and a 2-cycle misprediction penalty, which scheme is best: predict-taken, predict-not-taken, or 90% accurate dynamic?**
A: The predict-not-taken scheme is best, as it will be correct 95% of the time.
32. **Q: For a branch with 95% taken frequency and a 2-cycle misprediction penalty, which scheme is best: predict-taken, predict-not-taken, or 90% accurate dynamic?**
A: The predict-taken scheme is best, as it will be correct 95% of the time.

33. **Q: For a branch with 70% taken frequency and a 2-cycle misprediction penalty, which scheme is best: predict-taken, predict-not-taken, or 90% accurate dynamic?**
A: The dynamic predictor is best, as its 90% accuracy is higher than predict-taken (70%) or predict-not-taken (30%).
34. **Q: What is the name for a pipeline hazard involving conditional branches?**
A: It is called a control hazard or a branch hazard.
35. **Q: The delay in determining the proper instruction to fetch due to a branch instruction is called a _____ or branch hazard.**
A: control hazard
36. **Q: Why is the section on control hazards presented as shorter than the one on data hazards?**
A: Control hazards are simpler to understand, occur less frequently, and have less effective resolution schemes compared to forwarding for data hazards.
37. **Q: What is a simple static branch prediction scheme that improves upon stalling?**
A: The 'assume branch not taken' scheme, where execution continues down the sequential instruction stream.
38. **Q: What action is taken to discard instructions in a pipeline when a branch is mispredicted?**
A: The original control values for the incorrect instructions are changed to 0s.
39. **Q: Term: Flush**
A: Definition: To discard instructions in a pipeline, usually due to an unexpected event like a mispredicted branch.
40. **Q: In the unoptimized pipeline, which stages must be flushed when a branch taken in the MEM stage is discovered?**
A: The instructions in the IF, ID, and EX stages must be flushed.
41. **Q: What is one way to improve conditional branch performance by reducing the cost of a taken branch?**
A: Move the conditional branch execution earlier in the pipeline, such as to the ID stage.
42. **Q: How is the branch target address calculation moved to the ID stage?**
A: The branch adder is moved from the EX stage to the ID stage, using the PC and immediate field from the IF/ID register.
43. **Q: What is the more difficult part of moving branch execution to the ID stage?**
A: Moving the branch decision itself, which involves comparing two register values during the ID stage.
44. **Q: Moving the branch test to the ID stage introduces the need for new _____ and _____ hardware.**
A: forwarding, hazard detection
45. **Q: If branch comparison is moved to ID, from which pipeline registers can the bypassed source operands come?**
A: The bypassed operands can come from either the EX/MEM or MEM/WB pipeline registers.
46. **Q: If branch execution is moved to the ID stage, what happens if an ALU instruction immediately preceding a branch produces an operand for the branch test?**
A: A data hazard occurs, and a one-cycle stall is required.
47. **Q: If branch execution is moved to the ID stage, what is the penalty for a load instruction immediately followed by a branch that depends on the load result?**
A: A two-cycle stall is needed.
48. **Q: What is the primary benefit of moving conditional branch execution to the ID stage?**
A: It reduces the penalty of a taken branch to only one instruction.
49. **Q: What is the purpose of the IF.Flush control line?**
A: It zeros the instruction field of the IF/ID pipeline register to flush the instruction currently in the IF stage.

50. **Q: Clearing the IF/ID register with IF.Flush transforms the fetched instruction into a _____, an instruction that has no action.**
A: nop
51. **Q: What is the term for predicting branches at runtime using runtime information?**
A: Dynamic branch prediction.
52. **Q: Is a prediction from a simple branch prediction buffer guaranteed to be for the correct branch instruction?**
A: No, another branch with the same low-order address bits may have set the prediction bit, but this does not affect correctness.
53. **Q: What happens in a 1-bit dynamic prediction scheme when a prediction turns out to be wrong?**
A: The incorrectly predicted instructions are deleted, the prediction bit is inverted, and the proper sequence is fetched.
54. **Q: What is the performance shortcoming of a simple 1-bit prediction scheme, especially with loops?**
A: It can mispredict twice for a frequently-taken loop: once on the first iteration and once on the final, non-taken iteration.
55. **Q: For a loop branch that is taken 9 times and not taken once, what is the prediction accuracy of a 1-bit scheme?**
A: The accuracy is 80%, with two incorrect predictions (first and last iteration) and eight correct ones.
56. **Q: How does a 2-bit branch prediction scheme improve upon a 1-bit scheme?**
A: A prediction must be wrong twice before it is changed, making it more resilient to single changes in branch behavior.
57. **Q: In a 2-bit prediction scheme, a branch that strongly favors taken or not taken will be mispredicted how many times when its pattern is briefly interrupted?**
A: It will be mispredicted only once.
58. **Q: A 2-bit scheme is a general instance of a _____ predictor, which is incremented on an accurate prediction and decremented otherwise.**
A: counter-based
59. **Q: Term: Branch Target Buffer**
A: Definition: A structure that caches the destination PC or destination instruction for a branch to reduce or eliminate the taken-branch penalty.
60. **Q: What kind of predictor combines the local behavior of a branch with global information about recently executed branches?**
A: A correlating predictor.
61. **Q: Term: Correlating Predictor**
A: Definition: A branch predictor that combines local behavior of a particular branch and global information about the behavior of some recent number of executed branches.
62. **Q: Term: Tournament Branch Predictor**
A: Definition: A predictor with multiple prediction mechanisms for each branch and a selector that chooses which one to use based on which has been more accurate.
63. **Q: What is an architectural alternative to conditional branches that can reduce their frequency?**
A: Conditional move instructions, which conditionally change a destination register's value instead of the PC.
64. **Q: What is the function of the ARMv8 CSEL instruction?**
A: It conditionally selects one of two source registers to copy to a destination register based on the condition codes.
65. **Q: Given a 2-cycle misprediction penalty and 0-cycle correct prediction penalty, which scheme is best for a branch taken with 5% frequency?**
A: Predict not taken is best, as it is correct 95% of the time, yielding a lower average penalty than the 90% accurate dynamic predictor.

66. **Q: Given a 2-cycle misprediction penalty and 0-cycle correct prediction penalty, which scheme is best for a branch taken with 95% frequency?**
A: Predict taken is best, as it is correct 95% of the time, yielding a lower average penalty than the 90% accurate dynamic predictor.
67. **Q: Given a 2-cycle misprediction penalty and 0-cycle correct prediction penalty, which scheme is best for a branch taken with 70% frequency?**
A: Dynamic prediction is best, as its 90% accuracy is better than both predict taken (70%) and predict not taken (30%).

31 Exercises

Exercise 4.1

Consider the following instruction: Instruction: and rd, rs1, rs2 Interpretation: $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] \text{ AND } \text{Reg}[\text{rs2}]$

- 4.1.1 (5 pts) §4.3 What are the values of control signals generated by the control in Figure 4.10 for this instruction?
- 4.1.2 (5 pts) §4.3 Which resources (blocks) perform a useful function for this instruction?
- 4.1.3 (10 pts) §4.3 Which resources (blocks) produce no output for this instruction? Which resources produce output that is not used?

Exercise 4.2

[10] §4.4 Explain each of the “don’t cares” in Figure 4.18.

Exercise 4.3

Consider the following instruction mix:

- 4.3.1 (5 pts) §4.4 What fraction of all instructions use data memory?
- 4.3.2 (5 pts) §4.4 What fraction of all instructions use instruction memory?
- 4.3.3 (5 pts) §4.4 What fraction of all instructions use the sign extend?
- 4.3.4 (5 pts) §4.4 What is the sign extend doing during cycles in which its output is not needed?

Exercise 4.4

When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one signal wire to get “broken” and always register a logical 0. This is often called a “stuck-at-0” fault.

- 4.4.1 (5 pts) §4.4 Which instructions fail to operate correctly if the MemToReg wire is stuck at 0?
- 4.4.2 (5 pts) §4.4 Which instructions fail to operate correctly if the ALUSrc wire is stuck at 0?

Exercise 4.5

In this exercise, we examine in detail how an instruction is executed in a single-cycle datapath. Problems in this exercise refer to a clock cycle in which the processor fetches the following instruction word: 0x00c6ba23.

- 4.5.1 (10 pts) §4.4 What are the values of the ALU control unit’s inputs for this instruction?
- 4.5.2 (5 pts) §4.4 What is the new PC address after this instruction is executed? Highlight the path through which this value is determined.
- 4.5.3 (10 pts) §4.4 For each mux, show the values of its inputs and outputs during the execution of this instruction. List values that are register outputs at $\text{Reg}[\text{xn}]$.
- 4.5.4 (10 pts) §4.4 What are the input values for the ALU and the two add units?
- 4.5.5 (10 pts) §4.4 What are the values of all inputs for the registers unit?

Exercise 4.6

Section 4.4 does not discuss I-type instructions like `addi` or `andi`.

- 4.6.1 (5 pts) §4.4 What additional logic blocks, if any, are needed to add I-type instructions to the CPU shown in Figure 4.21? Add any necessary logic blocks to Figure 4.21 and explain their purpose.
- 4.6.2 (10 pts) §4.4 List the values of the signals generated by the control unit for `addi`. Explain the reasoning for any “don’t care” control signals.

Exercise 4.7

Problems in this exercise assume that the logic blocks used to implement a processor’s datapath have the following latencies: “Register read” is the time needed after the rising clock edge for the new register value to appear on the output. This value applies to the PC only. “Register setup” is the amount of time a register’s data input must be stable before the rising edge of the clock. This value applies to both the PC and Register File.

- 4.7.1 (5 pts) §4.4 What is the latency of an R-type instruction (i.e., how long must the clock period be to ensure that this instruction works correctly)?
- 4.7.2 (10 pts) §4.4 What is the latency of `ld`? (Check your answer carefully. Many students place extra muxes on the critical path.)
- 4.7.3 (10 pts) §4.4 What is the latency of `sd`? (Check your answer carefully. Many students place extra muxes on the critical path.)
- 4.7.4 (5 pts) §4.4 What is the latency of `beq`?
- 4.7.5 (5 pts) §4.4 What is the latency of an I-type instruction?
- 4.7.6 (5 pts) §4.4 What is the minimum clock period for this CPU?

Exercise 4.8

[10] §4.4 Suppose you could build a CPU where the clock cycle time was different for each instruction. What would the speedup of this new CPU be over the CPU presented in Figure 4.21 given the instruction mix below?

Exercise 4.9

Consider the addition of a multiplier to the CPU shown in Figure 4.21. This addition will add 300 ps to the latency of the ALU, but will reduce the number of instructions by 5% (because there will no longer be a need to emulate the `multiply` instruction).

- 4.9.1 (5 pts) §4.4 What is the clock cycle time with and without this improvement?
- 4.9.2 (10 pts) §4.4 What is the speedup achieved by adding this improvement?
- 4.9.3 (10 pts) §4.4 What is the slowest the new ALU can be and still result in improved performance?

Exercise 4.10

When processor designers consider a possible improvement to the processor datapath, the decision usually depends on the cost/performance trade-off. In the following three problems, assume that we are beginning with the datapath from Figure 4.21, the latencies from Exercise 4.7, and the following costs: Suppose doubling the number of general purpose registers from 32 to 64 would reduce the number of ld and sd instruction by 12%, but increase the latency of the register file from 150 ps to 160 ps and double the cost from 200 to 400. (Use the instruction mix from Exercise 4.8 and ignore the other effects on the ISA discussed in Exercise 2.18.)

- 4.10.1 (5 pts) §4.4 What is the speedup achieved by adding this improvement?
- 4.10.2 (10 pts) §4.4 Compare the change in performance to the change in cost.
- 4.10.3 (10 pts) §4.4 Given the cost/performance ratios you just calculated, describe a situation where it makes sense to add more registers and describe a situation where it doesn't make sense to add more registers.

Exercise 4.11

Examine the difficulty of adding a proposed lwi.d rd, rs1, rs2 ("Load With Increment") instruction to RISC-V. Interpretation: $\text{Reg}[\text{rd}] = \text{Mem}[\text{Reg}[\text{rs1}] + \text{Reg}[\text{rs2}]]$

- 4.11.1 (5 pts) §4.4 Which new functional blocks (if any) do we need for this instruction?
- 4.11.2 (5 pts) §4.4 Which existing functional blocks (if any) require modification?
- 4.11.3 (5 pts) §4.4 Which new data paths (if any) do we need for this instruction?
- 4.11.4 (5 pts) §4.4 What new signals do we need (if any) from the control unit to support this instruction?

Exercise 4.12

Examine the difficulty of adding a proposed swap rs1, rs2 instruction to RISC-V. Interpretation: $\text{Reg}[\text{rs2}] = \text{Reg}[\text{rs1}]; \text{Reg}[\text{rs1}] = \text{Reg}[\text{rs2}]$

- 4.12.1 (5 pts) §4.4 Which new functional blocks (if any) do we need for this instruction?
- 4.12.2 (10 pts) §4.4 Which existing functional blocks (if any) require modification?
- 4.12.3 (5 pts) §4.4 What new data paths do we need (if any) to support this instruction?
- 4.12.4 (5 pts) §4.4 What new signals do we need (if any) from the control unit to support this instruction?
- 4.12.5 (5 pts) §4.4 Modify Figure 4.21 to demonstrate an implementation of this new instruction.

Exercise 4.13

Examine the difficulty of adding a proposed ss rs1, rs2, imm (Store Sum) instruction to RISC-V. Interpretation: $\text{Mem}[\text{Reg}[\text{rs1}]] = \text{Reg}[\text{rs2}] + \text{immediate}$

- 4.13.1 (10 pts) §4.4 Which new functional blocks (if any) do we need for this instruction?
- 4.13.2 (10 pts) §4.4 Which existing functional blocks (if any) require modification?
- 4.13.3 (5 pts) §4.4 What new data paths do we need (if any) to support this instruction?
- 4.13.4 (5 pts) §4.4 What new signals do we need (if any) from the control unit to support this instruction?
- 4.13.5 (5 pts) §4.4 Modify Figure 4.21 to demonstrate an implementation of this new instruction.

Exercise 4.14

[5] §4.4: For which instructions (if any) is the Imm Gen block on the critical path?

Exercise 4.15

ld is the instruction with the longest latency on the CPU from Section 4.4. If we modified ld and sd so that there was no offset (i.e., the address to be loaded from/stored to must be calculated and placed in rs1 before calling ld/sd), then no instruction would use both the ALU and Data memory. This would allow us to reduce the clock cycle time. However, it would also increase the number of instructions, because many ld and sd instructions would need to be replaced with ld/add or sd/add combinations.

- 4.15.1 (5 pts) §4.4 What would the new clock cycle time be?
- 4.15.2 (10 pts) §4.4 Would a program with the instruction mix presented in Exercise 4.7 run faster or slower on this new CPU? By how much? (For simplicity, assume every ld and sd instruction is replaced with a sequence of two instructions.)
- 4.15.3 (5 pts) §4.4 What is the primary factor that influences whether a program will run faster or slower on the new CPU?
- 4.15.4 (5 pts) §4.4 Do you consider the original CPU (as shown in Figure 4.21) a better overall design; or do you consider the new CPU a better overall design? Why?

Exercise 4.16

In this exercise, we examine how pipelining affects the clock cycle time of the processor. Problems in this exercise assume that individual stages of the datapath have the following latencies: Also, assume that instructions executed by the processor are broken down as follows:

- 4.16.1 (5 pts) §4.5 What is the clock cycle time in a pipelined and non-pipelined processor?
- 4.16.2 (10 pts) §4.5 What is the total latency of an ld instruction in a pipelined and non-pipelined processor?
- 4.16.3 (10 pts) §4.5 If we can split one stage of the pipelined datapath into two new stages, each with half the latency of the original stage, which stage would you split and what is the new clock cycle time of the processor?
- 4.16.4 (10 pts) §4.5 Assuming there are no stalls or hazards, what is the utilization of the data memory?
- 4.16.5 (10 pts) §4.5 Assuming there are no stalls or hazards, what is the utilization of the write-register port of the “Registers” unit?

Exercise 4.17

[10] §4.5: What is the minimum number of cycles needed to completely execute n instructions on a CPU with a k stage pipeline? Justify your formula.

Exercise 4.18

[5] §4.5: Assume that x11 is initialized to 11 and x12 is initialized to 22. Suppose you executed the code below on a version of the pipeline from Section 4.5 that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). What would the final values of registers x13 and x14 be? `addix11, x12, 5` `addx13, x11, x12` `addix14, x11, 15`

Exercise 4.19

[10] §4.5 Assume that x11 is initialized to 11 and x12 is initialized to 22. Suppose you executed the code below on a version of the pipeline from Section 4.5 that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). What would the final values of register x15 be? Assume the register file is written at the beginning of the cycle and read at the end of a cycle. Therefore, an ID stage will return the results of a WB state occurring during the same cycle. See Section 4.7 and Figure 4.51 for details. `addxi11, x12, 5 addxi13, x11, x12 addxi14, x11, 15 addxi15, x11, x11`

Exercise 4.20

[5] §4.5 Add NOP instructions to the code below so that it will run correctly on a pipeline that does not handle data hazards. `addxi11, x12, 5 addxi13, x11, x12 addxi14, x11, 15 addxi15, x13, x12`

Exercise 4.21

Consider a version of the pipeline from Section 4.5 that does not handle data hazards (i.e., the programmer is responsible for addressing data hazards by inserting NOP instructions where necessary). Suppose that (after optimization) a typical n instruction program requires an additional $4*n$ NOP instructions to correctly handle data hazards.

- 4.21.1 (5 pts) §4.5 Suppose that the cycle time of this pipeline without forwarding is 250 ps. Suppose also that adding forwarding hardware will reduce the number of NOPs from $.4*n$ to $.05*n$, but increase the cycle time to 300 ps. What is the speedup of this new pipeline compared to the one without forwarding?
- 4.21.2 (10 pts) §4.5 Different programs will require different amounts of NOPs. How many NOPs (as a percentage of code instructions) can remain in the typical program before that program runs slower on the pipeline with forwarding?
- 4.21.3 (10 pts) §4.5 Repeat 4.21.2; however, this time let x represent the number of NOP instructions relative to n . (In 4.21.2, x was equal to $.4$.) Your answer will be with respect to x .
- 4.21.4 (10 pts) §4.5 Can a program with only $.075*n$ NOPs possibly run faster on the pipeline with forwarding? Explain why or why not.
- 4.21.5 (10 pts) §4.5 At minimum, how many NOPs (as a percentage of code instructions) must a program have before it can possibly run faster on the pipeline with forwarding?

Exercise 4.22

[5] §4.5 Consider the fragment of RISC-V assembly below: `sd x29, 12(x16) ld x29, 8(x16) sub x17, x15, x14 beqz x17, label add x15, x11, x14 sub x15, x30, x14` Suppose we modify the pipeline so that it has only one memory (that handles both instructions and data). In this case, there will be a structural hazard every time a program needs to fetch an instruction during the same cycle in which another instruction accesses data.

- 4.22.1 (5 pts) §4.5 Draw a pipeline diagram to show where the code above will stall.
- 4.22.2 (5 pts) §4.5 In general, is it possible to reduce the number of stalls/NOPs resulting from this structural hazard by reordering code?
- 4.22.3 (5 pts) §4.5 Must this structural hazard be handled in hardware? We have seen that data hazards can be eliminated by adding NOPs to the code. Can you do the same with this structural hazard? If so, explain how. If not, explain why not.
- 4.22.4 (5 pts) §4.5 Approximately how many stalls would you expect this structural hazard to generate in a typical program? (Use the instruction mix from Exercise 4.8.)

Exercise 4.23

If we change load/store instructions to use a register (without an offset) as the address, these instructions no longer need to use the ALU. (See Exercise 4.15.) As a result, the MEM and EX stages can be overlapped and the pipeline has only four stages.

- 4.23.1 (10 pts) §4.5 How will the reduction in pipeline depth affect the cycle time?
- 4.23.2 (5 pts) §4.5 How might this change improve the performance of the pipeline?
- 4.23.3 (5 pts) §4.5 How might this change degrade the performance of the pipeline?

Exercise 4.24

[10] §4.7 Which of the two pipeline diagrams below better describes the operation of the pipeline's hazard detection unit? Why? Choice 1: ld x11, 0(x12): IF ID EX ME WB add x13, x11, x14: IF ID EX..ME WB or x15, x16, x17: IF ID..EX ME WB Choice 2: ld x11, 0(x12): IF ID EX ME WB add x13, x11, x14: IF ID..EX ME WB or x15, x16, x17: IF..ID EX ME WB

Exercise 4.25

Consider the following loop. LOOP: ld x10, 0(x13) ld x11, 8(x13) add x12, x10, x11 sub x13, x13, 16 bnez x12, LOOP Assume that perfect branch prediction is used (no stalls due to control hazards), that there are no delay slots, that the pipeline has full forwarding support, and that branches are resolved in the EX (as opposed to the ID) stage.

- 4.25.1 (10 pts) §4.7 Show a pipeline execution diagram for the first two iterations of this loop.
- 4.25.2 (10 pts) §4.7 Mark pipeline stages that do not perform useful work. How often while the pipeline is full do we have a cycle in which all five pipeline stages are doing useful work? (Begin with the cycle during which the sub is in the IF stage. End with the cycle during which the bnez is in the IF stage.)

Exercise 4.26

This exercise is intended to help you understand the cost/complexity/performance trade-offs of forwarding in a pipelined processor. Problems in this exercise refer to pipelined datapaths from Figure 4.53. These problems assume that, of all the instructions executed in a processor, the following fraction of these instructions has a particular type of RAW data dependence. The type of RAW data dependence is identified by the stage that produces the result (EX or MEM) and the next instruction that consumes the result (1st instruction that follows the one that produces the result, 2nd instruction that follows, or both). We assume that the register write is done in the first half of the clock cycle and that register reads are done in the second half of the cycle, so "EX to 3rd" and "MEM to 3rd" dependences are not counted because they cannot result in data hazards. We also assume that branches are resolved in the EX stage (as opposed to the ID stage), and that the CPI of the processor is 1 if there are no data hazards. Assume the following latencies for individual pipeline stages. For the EX stage, latencies are given separately for a processor without forwarding and for a processor with different kinds of forwarding.

- 4.26.1 (5 pts) §4.7 For each RAW dependency listed above, give a sequence of at least three assembly statements that exhibits that dependency.
- 4.26.2 (5 pts) §4.7 For each RAW dependency above, how many NOPs would need to be inserted to allow your code from
- 4.26.1 (pts) to run correctly on a pipeline with no forwarding or hazard detection? Show where the NOPs could be inserted.
- 4.26.3 (10 pts) §4.7 Analyzing each instruction independently will over-count the number of NOPs needed to run a program on a pipeline with no forwarding or hazard detection. Write a sequence of three assembly instructions so that, when you consider each instruction in the sequence independently, the sum of the stalls is larger than the number of stalls the sequence actually needs to avoid data hazards.

- 4.26.4 (5 pts) §4.7 Assuming no other hazards, what is the CPI for the program described by the table above when run on a pipeline with no forwarding? What percent of cycles are stalls?
- 4.26.5 (5 pts) §4.7 What is the CPI if we use full forwarding (forward all results that can be forwarded)? What percent of cycles are stalls?
- 4.26.6 (10 pts) §4.7 Let us assume that we cannot afford to have three-input multiplexors that are needed for full forwarding. We have to decide if it is better to forward only from the EX/MEM pipeline register (next-cycle forwarding) or only from the MEM/WB pipeline register (two-cycle forwarding). What is the CPI for each option?
- 4.26.7 (5 pts) §4.7 For the given hazard probabilities and pipeline stage latencies, what is the speedup achieved by each type of forwarding (EX/MEM, MEM/WB, for full) as compared to a pipeline that has no forwarding?
- 4.26.8 (5 pts) §4.7 What would be the additional speedup (relative to the fastest processor from 4.26.7) be if we added “time travel” forwarding that eliminates all data hazards? Assume that the yet-to-be-invented time-travel circuitry adds 100 ps to the latency of the full-forwarding EX stage.
- 4.26.9 (5 pts) §4.7 The table of hazard types has separate entries for “EX to 1st” and “EX to 1st and EX to 2nd”. Why is there no entry for “MEM to 1st and MEM to 2nd”?

Exercise 4.27

Problems in this exercise refer to the following sequence of instructions, and assume that it is executed on a five-stage pipelined datapath: `add x15, x12, x11` `ld x13, 4(x15)` `ld x12, 0(x2)` `or x13, x15, x13` `sd x13, 0(x15)`

- 4.27.1 (5 pts) §4.7 If there is no forwarding or hazard detection, insert NOPs to ensure correct execution.
- 4.27.2 (10 pts) §4.7 Now, change and/or rearrange the code to minimize the number of NOPs needed. You can assume register x17 can be used to hold temporary values in your modified code.
- 4.27.3 (10 pts) §4.7 If the processor has forwarding, but we forgot to implement the hazard detection unit, what happens when the original code executes?
- 4.27.4 (20 pts) §4.7 If there is forwarding, for the first seven cycles during the execution of this code, specify which signals are asserted in each cycle by hazard detection and forwarding units in Figure 4.59.
- 4.27.5 (10 pts) §4.7 If there is no forwarding, what new input and output signals do we need for the hazard detection unit in Figure 4.59? Using this instruction sequence as an example, explain why each signal is needed.
- 4.27.6 (20 pts) §4.7 For the new hazard detection unit from 4.26.5, specify which output signals it asserts in each of the first five cycles during the execution of this code.

Exercise 4.28

The importance of having a good branch predictor depends on how often conditional branches are executed. Together with branch predictor accuracy, this will determine how much time is spent stalling due to mispredicted branches. In this exercise, assume that the breakdown of dynamic instructions into various instruction categories is as follows: Also, assume the following branch predictor accuracies: Always-Taken 45%, Always-Not-Taken 55%, 2-Bit 85%.

- 4.28.1 (10 pts) §4.8 Stall cycles due to mispredicted branches increase the CPI. What is the extra CPI due to mispredicted branches with the always-taken predictor? Assume that branch outcomes are determined in the ID stage and applied in the EX stage that there are no data hazards, and that no delay slots are used.
- 4.28.2 (10 pts) §4.8 Repeat
- 4.28.1 (pts) for the “always-not-taken” predictor.
- 4.28.3 (10 pts) §4.8 Repeat

- 4.28.1 (pts) for the 2-bit predictor.
- 4.28.4 (10 pts) §4.8 With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions to some ALU instruction? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.
- 4.28.5 (10 pts) §4.8 With the 2-bit predictor, what speedup would be achieved if we could convert half of the branch instructions in a way that replaced each branch instruction with two ALU instructions? Assume that correctly and incorrectly predicted instructions have the same chance of being replaced.
- 4.28.6 (10 pts) §4.8 Some branch instructions are much more predictable than others. If we know that 80% of all executed branch instructions are easy-to-predict loop-back branches that are always predicted correctly, what is the accuracy of the 2-bit predictor on the remaining 20% of the branch instructions?

Exercise 4.29

This exercise examines the accuracy of various branch predictors for the following repeating pattern (e.g., in a loop) of branch outcomes: T, NT, T, T, NT.

- 4.29.1 (5 pts) §4.8 What is the accuracy of always-taken and always-not-taken predictors for this sequence of branch outcomes?
- 4.29.2 (5 pts) §4.8 What is the accuracy of the 2-bit predictor for the first four branches in this pattern, assuming that the predictor starts off in the bottom left state from Figure 4.61 (predict not taken)?
- 4.29.3 (10 pts) §4.8 What is the accuracy of the 2-bit predictor if this pattern is repeated forever?
- 4.29.4 (30 pts) §4.8 Design a predictor that would achieve a perfect accuracy if this pattern is repeated forever. Your predictor should be a sequential circuit with one output that provides a prediction (1 for taken, 0 for not taken) and no inputs other than the clock and the control signal that indicates that the instruction is a conditional branch.
- 4.29.5 (10 pts) §4.8 What is the accuracy of your predictor from
- 4.29.4 (pts) if it is given a repeating pattern that is the exact opposite of this one?
- 4.29.6 (20 pts) §4.8 Repeat 4.29.4, but now your predictor should be able to eventually (after a warm-up period during which it can make wrong predictions) start perfectly predicting both this pattern and its opposite. Your predictor should have an input that tells it what the real outcome was. Hint: this input lets your predictor determine which of the two repeating patterns it is given.

Exercise 4.30

This exercise explores how exception handling affects pipeline design. The first three problems in this exercise refer to the following two instructions: Instruction 1: beqz x11, LABEL Instruction 2: ld x11, 0(x12)

- 4.30.1 (5 pts) §4.9 Which exceptions can each of these instructions trigger? For each of these exceptions, specify the pipeline stage in which it is detected.
- 4.30.2 (10 pts) §4.9 If there is a separate handler address for each exception, show how the pipeline organization must be changed to be able to handle this exception. You can assume that the addresses of these handlers are known when the processor is designed.
- 4.30.3 (10 pts) §4.9 If the second instruction is fetched immediately after the first instruction, describe what happens in the pipeline when the first instruction causes the first exception you listed in Exercise 4.30.1. Show the pipeline execution diagram from the time the first instruction is fetched until the time the first instruction of the exception handler is completed.
- 4.30.4 (20 pts) §4.9 In vectored exception handling, the table of exception handler addresses is in data memory at a known (fixed) address. Change the pipeline to implement this exception handling mechanism. Repeat Exercise
- 4.30.3 (pts) using this modified pipeline and vectored exception handling.

- 4.30.5 (15 pts) §4.9 We want to emulate vectored exception handling (described in Exercise 4.30.4) on a machine that has only one fixed handler address. Write the code that should be at that fixed address. Hint: this code should identify the exception, get the right address from the exception vector table, and transfer execution to that handler.

Exercise 4.31

In this exercise we compare the performance of 1-issue and 2 issue processors, taking into account program transformations that can be made to optimize for 2-issue execution. Problems in this exercise refer to the following loop (written in C): `for(i=0;i!=j;i+=2) b[i]=a[i]-a[i+1]`; A compiler doing little or no optimization might produce the following RISC-V assembly code: `li x12, 0 jal ENT TOP: slli x5, x12, 3 add x6, x10, x5 ld x7, 0(x6) ld x29, 8(x6) sub x30, x7, x29 add x31, x11, x5 sd x30, 0(x31) addi x12, x12, 2 ENT: bne x12, x13, TOP` The code above uses the following registers: Assume the two-issue, statically scheduled processor for this exercise has the following properties: 1. One instruction must be a memory operation; the other must be an arithmetic/logic instruction or a branch. 2. The processor has all possible forwarding paths between stages (including paths to the ID stage for branch resolution). 3. The processor has perfect branch prediction. 4. Two instruction may not issue together in a packet if one depends on the other. (See page 324.) 5. If a stall is necessary, both instructions in the issue packet must stall. (See page 324.)

- 4.31.1 (30 pts) §4.10 Draw a pipeline diagram showing how RISC-V code given above executes on the two-issue processor. Assume that the loop exits after two iterations.
- 4.31.2 (10 pts) §4.10 What is the speedup of going from a one-issue to a two-issue processor? (Assume the loop runs thousands of iterations.)
- 4.31.3 (10 pts) §4.10 Rearrange/rewrite the RISC-V code given above to achieve better performance on the one-issue processor. Hint: Use the instruction “`beqz x13,DONE`” to skip the loop entirely if `j = 0`.
- 4.31.4 (20 pts) §4.10 Rearrange/rewrite the RISC-V code given above to achieve better performance on the two-issue processor. (Do not unroll the loop, however.)
- 4.31.5 (30 pts) §4.10 Repeat Exercise 4.31.1, but this time use your optimized code from Exercise 4.31.4.
- 4.31.6 (10 pts) §4.10 What is the speedup of going from a one-issue processor to a two-issue processor when running the optimized code from Exercises
- 4.31.3 (pts) and 4.31.4.
- 4.31.7 (10 pts) §4.10 Unroll the RISC-V code from Exercise
- 4.31.3 (pts) so that each iteration of the unrolled loop handles two iterations of the original loop. Then, rearrange/rewrite your unrolled code to achieve better performance on the one-issue processor. You may assume that `j` is a multiple of 4.
- 4.31.8 (20 pts) §4.10 Unroll the RISC-V code from Exercise
- 4.31.4 (pts) so that each iteration of the unrolled loop handles two iterations of the original loop. Then, rearrange/rewrite your unrolled code to achieve better performance on the two-issue processor. You may assume that `j` is a multiple of 4. (Hint: Re-organize the loop so that some calculations appear both outside the loop and at the end of the loop. You may assume that the values in temporary registers are not needed after the loop.)
- 4.31.9 (10 pts) §4.10 What is the speedup of going from a one-issue processor to a two-issue processor when running the unrolled, optimized code from Exercises
- 4.31.7 (pts) and 4.31.8?
- 4.31.10 (30 pts) §4.10 Repeat Exercises
- 4.31.8 (pts) and 4.31.9, but this time assume the two-issue processor can run two arithmetic/logic instructions together. (In other words, the first instruction in a packet can be any type of instruction, but the second must be an arithmetic or logic instruction. Two memory operations cannot be scheduled at the same time.)

Exercise 4.32

This exercise explores energy efficiency and its relationship with performance. Problems in this exercise assume the following energy consumption for activity in Instruction memory, Registers, and Data memory. You can assume that the other components of the datapath consume a negligible amount of energy. (“Register Read” and “Register Write” refer to the register file only.) Assume that components in the datapath have the following latencies. You can assume that the other components of the datapath have negligible latencies.

- 4.32.1 (5 pts) §§4.3, 4.6, 4.14 How much energy is spent to execute an add instruction in a single-cycle design and in the five-stage pipelined design?
- 4.32.2 (10 pts) §§4.6, 4.14 What is the worst-case RISC-V instruction in terms of energy consumption? What is the energy spent to execute it?
- 4.32.3 (10 pts) §§4.6, 4.14 If energy reduction is paramount, how would you change the pipelined design? What is the percentage reduction in the energy spent by an ld instruction after this change?
- 4.32.4 (10 pts) §§4.6, 4.14 What other instructions can potentially benefit from the change discussed in Exercise 4.32.3?
- 4.32.5 (10 pts) §§4.6, 4.14 How do your changes from Exercise 4.32.3 (pts) affect the performance of a pipelined CPU?
- 4.32.6 (10 pts) §§4.6, 4.14 We can eliminate the MemRead control signal and have the data memory be read in every cycle, i.e., we can permanently have MemRead=1. Explain why the processor still functions correctly after this change. If 25% of instructions are loads, what is the effect of this change on clock frequency and energy consumption?

Exercise 4.33

When silicon chips are fabricated, defects in materials (e.g., silicon) and manufacturing errors can result in defective circuits. A very common defect is for one wire to affect the signal in another. This is called a “cross-talk fault”. A special class of cross-talk faults is when a signal is connected to a wire that has a constant logical value (e.g., a power supply wire). These faults, where the affected signal always has a logical value of either 0 or 1 are called “stuck-at-0” or “stuck-at-1” faults. The following problems refer to bit 0 of the Write Register input on the register file in Figure 4.21.

- 4.33.1 (10 pts) §§4.3, 4.4 Let us assume that processor testing is done by (1) filling the PC, registers, and data and instruction memories with some values (you can choose which values), (2) letting a single instruction execute, then (3) reading the PC, memories, and registers. These values are then examined to determine if a particular fault is present. Can you design a test (values for PC, memories, and registers) that would determine if there is a stuck-at-0 fault on this signal?
- 4.33.2 (10 pts) §§4.3, 4.4 Repeat Exercise 4.33.1 for a stuck-at-1 fault. Can you use a single test for both stuck-at-0 and stuck at-1? If yes, explain how; if no, explain why not.
- 4.33.3 (10 pts) §§4.3, 4.4 If we know that the processor has a stuck-at-1 fault on this signal, is the processor still usable? To be usable, we must be able to convert any program that executes on a normal RISC-V processor into a program that works on this processor. You can assume that there is enough free instruction memory and data memory to let you make the program longer and store additional data.
- 4.33.4 (10 pts) §§4.3, 4.4 Repeat Exercise 4.33.1; but now the fault to test for is whether the MemRead control signal becomes 0 if the branch control signal is 0, no fault otherwise.
- 4.33.5 (10 pts) §§4.3, 4.4 Repeat Exercise 4.33.1; but now the fault to test for is whether the MemRead control signal becomes 1 if RegRd control signal is 1, no fault otherwise. Hint: This problem requires knowledge of operating systems. Consider what causes segmentation faults.

32 Exercises

33 Answers to Exercises

Exercise 4.1

Instruction: and rd, rs1, rs2

Interpretation: $\text{Reg}[\text{rd}] = \text{Reg}[\text{rs1}] \text{ AND } \text{Reg}[\text{rs2}]$

4.1.1 Control Signals:

- $\text{RegWrite} = 1$ (Write to rd)
- $\text{ALUSrc} = 0$ (Second operand from register, not immediate)
- $\text{MemRead} = 0$
- $\text{MemWrite} = 0$
- $\text{MemtoReg} = 0$ (Result comes from ALU)
- $\text{Branch} = 0$
- $\text{ALUOp} = 10$ (R-type, function determined by funct field)

4.1.2 Useful Resources:

- Instruction Memory (Fetch instruction)
- Register File (Read rs1, rs2; Write rd)
- ALU (Perform AND operation)
- PC / Adder (Calculate next PC)
- Control Unit (Generate signals)

4.1.3 Unused/No Output:

- **No Output:** Data Memory ($\text{MemRead}=0$, so no valid output relative to instruction, though physically might produce garbage if address lines toggle, but logically unused).
- **Output Not Used:** Sign Extension unit produces output (for I-type fields), but it is ignored by the ALUSrc mux. Branch Adder produces a target address, but it is ignored by the PC mux ($\text{Branch}=0$). Data Memory output is ignored by MemtoReg mux.

Exercise 4.2

Don't Cares in Control:

- **MemtoReg for Stores:** Implementation dependent, but usually X because RegWrite is 0, so it doesn't matter what data is presented to the register file's write port; nothing will be written.
- **ALUOp for Loads/Stores:** For loads/stores, the ALU performs addition ($\text{base} + \text{offset}$). The ALUOp is set to '00' (Add). It is not a don't care.
- **Branch for Non-Branches:** Signal is 0.
- *Note: Specific "Don't Cares" usually refer to signals that don't affect state for the specific instruction. For example, for a beq, MemtoReg is X because RegWrite is 0. For an R-type, $\text{MemRead}/\text{MemWrite}$ are 0.*

Exercise 4.3

Note: The specific instruction mix table was not provided in the input text. Definitions provided below.

4.3.1 Data Memory Fraction: Sum of percentages for Loads and Stores.

4.3.2 Instruction Memory Fraction: 100% (Every instruction is fetched).

4.3.3 Sign Extend Fraction: Used by I-type (arithmetic immediate, loads), S-type (stores), SB-type (branches). Effectively 100% of instructions use the sign extender (or immediate generator) in RISC-V to prepare immediate values, even if unused by ALUSrc (like R-type). But functionally "used" implies the immediate is selected.

4.3.4 Idle Sign Extend: It continues to generate sign-extended values based on the instruction bits connected to its input. These values are simply blocked by multiplexers (e.g., ALUSrc Mux) and not used in the datapath.

Exercise 4.4

Stuck-at-0 Faults:

4.4.1 MemToReg stuck at 0:

- This signal selects between ALU Result (0) and Memory Output (1).
- **Stuck at 0 means:** Always selects ALU Result.
- **Failing Instructions:** Load instructions (**ld**, **lw**, etc.) will write the calculated address (from ALU) into the destination register instead of the data from memory.

4.4.2 ALUSrc stuck at 0:

- This signal selects between Register 2 (0) and Immediate (1).
- **Stuck at 0 means:** Always selects Register 2.
- **Failing Instructions:** All I-type ALUs (**addi**, **andi**, etc.), Loads (**ld**), Stores (**sd**). These require the immediate offset/value. They will instead use the value in rs2.

Exercise 4.5

Instruction Word: 0x00c6ba23

- Binary: 0000 0000 1100 0110 1011 1010 0010 0011
- Decoding:
 - Opcode (lowest 7 bits): 0100011 (Store instruction)
 - Funct3 (bits 12-14): 010 (**sw** - Store Word)
 - rs1 (bits 15-19): 01101 (13, **x13**)
 - rs2 (bits 20-24): 01100 (12, **x12**)
 - imm[11:5] (top 7): 0000000
 - imm[4:0] (bits 7-11): 00000
 - Total Immediate: 0
- **Instruction:** **sw x12, 0(x13)**

4.5.1 ALU Control Inputs:

- ALUOp from main control: 00 (Add for address calculation)
- ALU Control functionality: Add

4.5.2 New PC: $PC + 4$. Path: $PC \rightarrow \text{Adder (+4)} \rightarrow \text{PC Mux (Select 0)} \rightarrow PC$.

4.5.3 Mux Values:

- **ALUSrc Mux:** Selects Immediate (1). Input 0: Reg[x12], Input 1: 0 (Imm). Output: 0.
- **MemtoReg Mux:** Don't care (RegWrite=0).
- **PC Mux:** Selects PC+4 (0).

4.5.4 ALU Inputs:

- Input A: Reg[x13] (Base address)
- Input B: 0 (Immediate offset)
- Result: Reg[x13]

4.5.5 Registers Unit Inputs:

- Read Reg 1: x13
- Read Reg 2: x12
- Write Reg: x12 (bits 7-11 are passed, but Write Enable is 0)
- Write Data: Don't care (from MemtoReg mux)
- RegWrite: 0

Exercise 4.6

I-Type Instructions (addi):

4.6.1 Logic Blocks: The standard schematic usually supports I-types via the ALUSrc Mux (selecting immediate) and ImmGen unit. No *new* blocks are typically needed if the figure is the standard RISC-V single-cycle datapath (which includes ALUSrc mux). We just need accurate control.

4.6.2 Control Signals for addi:

- ALUSrc = 1 (Use Immediate)
- MemtoReg = 0 (Use ALU result)
- RegWrite = 1 (Write to rd)
- MemRead = 0
- MemWrite = 0
- Branch = 0
- ALUOp = 10 (or specific encoding for Add, typically I-type uses arithmetic op).

Exercise 4.7

Note: Specific latencies were not provided in the text input. Assuming standard conceptual latencies: Mem > ALU > RegFile.

Exercise 4.8 - 4.10

Note: Missing instruction mix/latency data tables.

Exercise 4.11

Instruction: `lwi.d rd, rs1, rs2`

Interpretation: $\text{Reg}[\text{rd}] = \text{Mem}[\text{Reg}[\text{rs1}] + \text{Reg}[\text{rs2}]]$

4.11.1 New Blocks: None.

4.11.2 Modifications: We need ALUSrc to select Read Data 2 (Reg[rs2]) instead of Immediate for a Load instruction. Standard Loads use Immediate. We might need to modify the ALUSrc Mux control or logic to allow "Reg+Reg" addressing for a Load.

4.11.3 New Datapaths: Path from Read Data 2 to ALU input B is already there (Input 0 of Mux). Just need to select it during a Load-like operation.

4.11.4 Control Signals: Need a signal to set ALUSrc=0 (Reg) while MemRead=1. Standard `ld` has ALUSrc=1.

Exercise 4.12

Instruction: `swap rs1, rs2`

Interpretation: $\text{Reg}[\text{rs2}] = \text{Reg}[\text{rs1}]; \text{Reg}[\text{rs1}] = \text{Reg}[\text{rs2}]$

Problem: This requires writing two registers in one cycle.

4.12.1 New Blocks: Need a second Write Port on the Register File.

4.12.2 Modifications: Register File needs 2 Write inputs (Data, Addr) and 2 Write Enables.

4.12.3 Datapaths:

- Path from Reg[rs1] (ReadData1) to WriteData2.
- Path from Reg[rs2] (ReadData2) to WriteData1.
- (Assuming swapping values directly, bypassing ALU).

4.12.4 Control: Two RegWrite signals.

4.12.5 Schematic: Major change to Register File ports.

Exercise 4.13

Instruction: `ss rs1, rs2, imm` (Store Sum)

Interpretation: $\text{Mem}[\text{Reg}[\text{rs1}]] = \text{Reg}[\text{rs2}] + \text{immediate}$

- Normal Store: $\text{Mem}[\text{rs1} + \text{imm}] = \text{rs2}$.
- This Store: Address = rs1. Data = rs2 + imm.

4.13.1 New Blocks: We need to add the immediate to rs2 before sending it to memory data input. Currently, ALU does address calculation.

4.13.2 Modifications:

- The ALU is busy calculating the address? No, Address is just rs1 (or rs1+0).
- We CAN use the ALU to calculate (rs2 + imm) if we bypass address calc.
- But Store Instruction usually uses ALU for Address = rs1 + imm. Here Address = rs1.
- So: ALU calculates (rs2 + imm). Result goes to Memory Write Data.
- Address comes directly from rs1 (Read Data 1).

4.13.3 Datapaths:

- Path from Read Data 1 directly to Memory Address (bypass ALU).
- Path from ALU Result to Memory Write Data (standard path is usually Read Data 2 to MemData, ALU to MemAddr).
- This essentially "reverses" the roles of ALU and direct path for Stores.
- Need Mux at Memory Address input: Select ALU Result (std) or Read Data 1 (ss).
- Need Mux at Memory Data input: Select Read Data 2 (std) or ALU Result (ss).

Exercise 4.14

Critical Path: The Imm Gen block is parallel to Register Read. It is usually not the standalone critical element, but it is part of the Instruction Decode phase.

- For `beq`, path is $\text{PC} \rightarrow \text{Mem} \rightarrow \text{ImmGen} \rightarrow \text{Shift} \rightarrow \text{Add} \rightarrow \text{Mux} \rightarrow \text{PC}$.
- For `ld`, path involves $\text{ImmGen} \rightarrow \text{ALU} \rightarrow \text{Mem}$.
- Yes, it is on the path for I-type, S-type, B-type, U-type, J-type. It is NOT used for R-type.

Exercise 4.15

Remove Offset from Load/Store: `'ld rd, (rs1)'` instead of `'ld rd, imm(rs1)'`.

4.15.1 Clock Cycle: Original Critical Path: Fetch + Decode + ALU + Mem + WB.

1. New Path: No ALU for Load. Fetch + Decode + Mem + WB.
2. This reduces cycle time by the latency of the ALU (approx).

4.15.2 Performance:

3. Code expansion: Every `'ld x, 4(y)'` becomes `'addi y, y, 4'` then `'ld x, 0(y)'`.
4. Instructions increase. Cycle time decreases.
5. Net result typically slower because instruction count increase (freq of loads/stores approx 30%) outweighs ALU latency removal (which is maybe 20% of path).

4.15.4 Design: Steps backwards. Addressing modes are powerful. Discarding offset makes pointers painful.

34 Exercises

Exercise 4.17

Question: Minimum cycles to execute n instructions on k stage pipeline?

1. **Formula:** $Cycles = n + (k - 1)$
2. **Justification:**
 - The first instruction takes k cycles to fill the pipeline and complete.
 - Each of the remaining $(n - 1)$ instructions completes 1 cycle after the previous one (ideal throughput).
 - Total = $k + (n - 1)$.

Exercise 4.23

Modified Pipeline (4 stages):

- 4.23.1 Effect on Cycle Time:** The cycle time is determined by the slowest stage. If we merge operations or change the critical path such that address calculation is removed (register direct), we might shorten the cycle time *if* the ALU was the bottleneck. However, if MEM access is slow, removing the ALU stage doesn't speed up MEM. If we overlap, we might make a stage longer.
- 4.23.2 Improvement:** Reduced pipeline depth ($k = 4$) reduces the branch misprediction penalty (flushing fewer stages) and reduces load-use latency penalties.
- 4.23.3 Degradation:** Using a register as address without offset removes the powerful "Base + Offset" addressing mode, forcing significant code expansion (extra ADD instructions to calculate addresses), which increases instruction count (n) and potentially slows down total execution time despite a possibly faster clock or shorter pipeline.

Exercise 4.30

Exception Handling:

- 4.30.1 Exceptions:**
- beqz: Invalid Instruction (ID), Misaligned Instruction Fetch (IF - though usually earlier).
 - ld: Invalid Address (EX/MEM), TLB Miss, Protection Fault.
- 4.30.2 Pipeline Changes:** Need a Cause Register to record the exception type. Need logic in each stage to detect valid exceptions and "kill" (flush) subsequent instructions. Need a mechanism to force the PC to the Trap Vector.

Exercise 4.33

Testing Manufacturing Faults (Stuck-at-0 on WriteReg Bit 0):

4.33.1 Test for Stuck-at-0:

- **Hypothesis:** If Bit 0 is stuck at 0, writing to an odd register (e.g., x1: ...001) will actually obtain the destination address of the even register below it (e.g., x0: ...000).
- **Test Sequence:**

```

addi x2, x0, 10    // Initialize x2 = 10
addi x3, x0, 99    // Write 99 to x3 (Binary ...011)
                  // If stuck-at-0, this writes to x2 (...010)

```
- **Verification:** Check the value of x2.
- **Result:** If x2 is 99, the fault exists. If x2 is 10 and x3 is 99, the fault does not exist.